

Assignment 3

FUMINI HOTEL MANAGEMENT SYSTEM (A3)

1.	Thông tin project.....	3
1.1.	Tên dự án	3
1.2.	Thông tin tác giả.....	3
2.	CẤU TRÚC PROJECT.....	4
2.1.1.	2.1 Backend (3-Layer Architecture).....	4
2.1.2.	2.2 Frontend (Feature-based Structure).....	6
3.	CHI TIẾT BACKEND API (CONTROLLERS).....	8
3.1.	Authentication.....	8
3.1.1.	CorsConfig.....	8
3.1.2.	SecurityConfig.....	9
3.1.3.	WebConfiguration.....	10
3.1.4.	JwtProperties.....	11
3.1.5.	JwtTokenProvider.....	12
3.1.6.	JwtTokenValidationFilter.....	14
3.1.7.	Result.....	16
3.2.	Customer Management.....	17
3.2.1.	Customer.....	17
3.2.2.	CustomerRepository.....	18
3.2.3.	CustomerServiceImpl.....	19
3.2.4.	CustomerController.....	20
3.2.5.	Result.....	20
3.3.	Quản lý phòng (RoomInformationController).....	22
3.3.1.	RoomInformation.....	22
3.3.2.	RoomType.....	23
3.3.3.	RoomInformationRepository.....	23
3.3.4.	RoomInformationService.....	24

3.3.5.	RoomInformationController.....	25
3.3.6.	Result.....	26
3.4.	Quản lý Đặt phòng (BookingReservationController)	27
3.4.1.	BookingReservation	27
3.4.2.	BookingReservationRepository	28
3.4.3.	BookingReservationServiceImpl	28
3.4.4.	BookingReservationController	29
3.4.5.	Result.....	30
4.	FRONTEND: DỊCH VỤ & HIỂN THỊ DỮ LIỆU.....	32
4.1.	Cấu hình Core & Bảo mật (Core Configuration).....	32
4.1.1.	axiosInstance.js (Kết nối API).....	32
4.1.2.	apiClient.js (Interceptors & JWT).....	33
4.1.3.	AuthContext.jsx (Quản lý trạng thái toàn cục).....	34
4.2.	Customer Management	41
4.2.1.	CustomerType.....	41
4.2.2.	CustomerService.....	42
4.2.3.	CustomerTable	43
4.2.4.	CustomerFormModal.....	44
4.2.5.	CustomerPages.....	45
4.3.	Feature: Quản lý Phòng (Room Management).....	47
4.3.1.	roomService.js.....	47
4.3.2.	RoomCard.jsx & RoomFilter.jsx (Customer View).....	49
4.3.3.	RoomFormModal.....	49
4.3.4.	RoomPage.....	50
4.4.	Feature: Quản lý Đặt phòng (Booking Management).....	50
4.4.1.	bookingService.js.....	50
4.4.2.	BookingFormModal.jsx.....	51
4.4.3.	. BookingTable.jsx (Staff View).....	54
4.4.4.	BookingHistoryPage.jsx.....	55
4.5.	Profile Page.....	57
4.6.	Other.....	58
4.6.1.	Header	58

4.6.2.	Footer.....	59
--------	-------------	----

1. Thông tin project

1.1. Tên dự án

FUMINI HOTEL MANAGEMENT SYSTEM (A3)

Link GitHub: <https://github.com/Michaelht271/SBA301-Spring-Boot-Application-With-ReactJS/tree/main/assignment/assignment3>

1.2. Thông tin tác giả

FullName: Nguyễn Văn An

Class: SE18D04

Subject: SBA301

Công nghệ: Spring Boot 3 + ReactJS + MS SQL Server

2. CẤU TRÚC PROJECT

2.1.1. 2.1 Backend (3-Layer Architecture)

```
michael@An:~/code/SBA301-Spring-Boot-Application-With-ReactJS/assigment/assignment3/A3NguyenV
├── mvnw
├── mvnw.cmd
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── michael
│   │   │   │   │   ├── a3nguyenvanan18d04
│   │   │   │   │   │   ├── A3NguyenVanAn18D04Application.java
│   │   │   │   │   │   ├── configs
│   │   │   │   │   │   │   ├── CorsConfig.java
│   │   │   │   │   │   │   ├── SecurityConfig.java
│   │   │   │   │   │   │   └── WebConfiguration.java
│   │   │   │   │   │   ├── controllers
│   │   │   │   │   │   │   ├── AuthController.java
│   │   │   │   │   │   │   ├── BookingDetailController.java
│   │   │   │   │   │   │   ├── BookingReservationController.java
│   │   │   │   │   │   │   ├── CustomerController.java
│   │   │   │   │   │   │   ├── RoomInformationController.java
│   │   │   │   │   │   │   └── RoomTypeController.java
│   │   │   │   │   │   ├── dtos
│   │   │   │   │   │   │   ├── auth
│   │   │   │   │   │   │   │   ├── LoginRequest.java
│   │   │   │   │   │   │   │   └── RegisterRequest.java
│   │   │   │   │   │   │   ├── booking
│   │   │   │   │   │   │   │   ├── request
│   │   │   │   │   │   │   │   │   ├── BookingDetailRequestDTO.java
│   │   │   │   │   │   │   │   │   ├── BookingRequestDTO.java
│   │   │   │   │   │   │   │   │   └── BookingStatusUpdatedDTO.java
│   │   │   │   │   │   │   └── rooms
│   │   │   │   │   │   │       ├── request
│   │   │   │   │   │   │       │   └── UpdateRoomRequest.java
│   │   │   │   │   │   ├── entites
│   │   │   │   │   │   │   ├── BookingDetail.java
│   │   │   │   │   │   │   ├── BookingReservation.java
│   │   │   │   │   │   │   ├── Customer.java
│   │   │   │   │   │   │   ├── RoomInformation.java
│   │   │   │   │   │   │   └── RoomType.java
│   │   │   │   │   │   ├── enums
│   │   │   │   │   │   │   ├── BookingStatus.java
│   │   │   │   │   │   │   ├── CustomerStatus.java
│   │   │   │   │   │   │   ├── Role.java
│   │   │   │   │   │   │   └── RoomStatus.java
```

```
├── enums
│   ├── BookingStatus.java
│   ├── CustomerStatus.java
│   ├── Role.java
│   └── RoomStatus.java
├── jwt
│   ├── JwtProperties.java
│   ├── JwtTokenProvider.java
│   └── JwtTokenValidationFilter.java
├── repository
│   ├── BookingDetailRepository.java
│   ├── BookingReservationRepository.java
│   ├── CustomerRepository.java
│   ├── RoomInformationRepository.java
│   └── RoomTypeRepository.java
├── services
│   ├── impl
│   │   ├── BookingDetailServiceImpl.java
│   │   ├── BookingReservationServiceImpl.java
│   │   ├── CustomerServiceImpl.java
│   │   ├── RoomInformationServiceImpl.java
│   │   └── RoomTypeServiceImpl.java
│   └── interfaces
│       ├── BookingDetailService.java
│       ├── BookingReservationService.java
│       ├── CustomerService.java
│       ├── RoomInformationService.java
│       └── RoomTypeService.java
├── utils
│   └── DataInitializer.java
├── resources
│   └── application.yml
├── test
│   ├── java
│   │   ├── com
│   │   │   ├── michael
│   │   │   │   ├── a3nguyenvanan18d04
│   │   │   │   │   ├── A3NguyenVanAn18D04ApplicationTests.java
│   │   │   │   │   └── services
│   │   │   │   │       ├── impl
│   │   │   │   │       │   └── RoomInformationServiceTest.java
```

2.1.2. 2.2 Frontend (Feature-based Structure)

```
michael@An:~/code/SBA301-Spring-Boot-Application-With-ReactJS/assignment/assignment3/a3NguyenVanAn18D04$ tree
.
├── eslint.config.js
├── index.html
├── package.json
├── package-lock.json
├── public
├── README.md
├── src
│   ├── App.css
│   ├── App.jsx
│   ├── assets
│   ├── core
│   │   ├── api
│   │   │   ├── apiClient.js
│   │   │   └── axiosInstance.js
│   │   ├── auth
│   │   │   ├── AuthContext.jsx
│   │   │   ├── authService.js
│   │   │   ├── jwtUtils.js
│   │   │   └── useAuth.js
│   │   ├── constants
│   │   │   └── index.js
│   │   └── utils
│   ├── features
│   │   ├── auth
│   │   │   ├── components
│   │   │   │   ├── LoginForm.jsx
│   │   │   │   └── RegisterForm.jsx
│   │   ├── bookings
│   │   │   ├── bookingService.js
│   │   │   ├── booking.type.js
│   │   │   ├── components
│   │   │   │   ├── BookingDetailModal.jsx
│   │   │   │   ├── BookingFormModal.jsx
│   │   │   │   └── BookingTable.jsx
│   │   │   └── useBookings.js
│   │   ├── customers
│   │   │   ├── components
│   │   │   │   ├── CustomerFormModal.jsx
│   │   │   │   └── CustomerTable.jsx
│   │   │   ├── customerService.js
│   │   │   └── customer.type.js
│   │   ├── profile
│   │   │   ├── components
│   │   │   │   ├── ProfileCard.jsx
│   │   │   │   └── ProfileForm.jsx
│   │   │   └── useProfile.js
│   └── rooms
```

```
├── useProfile.js
├── rooms
│   ├── components
│   │   ├── RoomCard.jsx
│   │   ├── RoomFilter.jsx
│   │   ├── RoomFormModal.jsx
│   │   └── RoomTable.jsx
│   ├── roomService.js
│   ├── room.type.js
│   ├── roomType.type.js
│   └── useRooms.js
├── hooks
├── index.css
├── main.jsx
├── pages
│   ├── auth
│   │   ├── LoginPage.jsx
│   │   └── RegisterPage.jsx
│   ├── customer
│   │   ├── BookingHistoryPage.jsx
│   │   ├── BookingPage.jsx
│   │   └── ProfilePage.jsx
│   ├── public
│   │   └── HomePage.jsx
│   └── staff
│       ├── BookingsPage.jsx
│       ├── CustomersPage.jsx
│       ├── DashboardPage.jsx
│       └── RoomsPage.jsx
├── routes
│   └── AppRouter.jsx
├── shared
│   ├── components
│   │   └── common
│   │       ├── Footer.jsx
│   │       ├── Header.jsx
│   │       └── PrivateRoute.jsx
│   ├── ui
│   └── layouts
│       ├── CustomerLayout.jsx
│       ├── PublicLayout.jsx
│       └── StaffLayout.jsx
├── vite.config.js
└── yarn.lock
```

32 directories, 58 files

michael@An:~/code/SBA301-Spring-Boot-Application-With-ReactJS/assigment/assignment3/a3NguyenVanAn18D04\$

a3NguyenVanAn18D04

3. CHI TIẾT BACKEND API (CONTROLLERS)

3.1. Authentication

3.1.1. CorsConfig

```
12 public class CorsConfig {
13
14     public CorsConfigurationSource corsConfigurationSource() {
15         CorsConfiguration corsConfiguration = new CorsConfiguration();
16
17         // 1. Allowed Origin
18         corsConfiguration.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:5173"));
19
20         // 2. Allowed Method
21
22         corsConfiguration.setAllowedMethods(List.of("POST", "GET", "PUT", "DELETE", "OPTIONS"));
23
24         // 3. Allowed Header
25         corsConfiguration.setAllowedHeaders(List.of(
26             "Authorization",
27             "Accept",
28             "X-Request-With",
29             "Content-Type",
30             "Access-Control-Request-Method",
31             "Access-Control-Request-Header"));
32
33         // 4. Exposed Headers
34         corsConfiguration.setExposedHeaders(List.of(
35             "Origin",
36             "Content-Type",
37             "Accept",
38             "Authorization",
39             "Access-Control-Allow-Origin",
40             "Access-Control-Allow-Credentials"
41         ));
42
43         // 5. Credentials
44         corsConfiguration.setAllowCredentials(true);
45
46         // 6. Cache
47         corsConfiguration.setMaxAge(3600L);
48
49         UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
50         source.registerCorsConfiguration(pattern: "**", corsConfiguration);
51         return source;
52     }
53 }
```

3.1.2. SecurityConfig

```
@Configuration
@EnableWebSecurity
@Slf4j
public class SecurityConfig {
    private final JwtProperties jwtProperties; 3 usages

    > public SecurityConfig(JwtProperties jwtProperties) { this.jwtProperties = jwtProperties; }

    @Bean
    > public SecretKey secretKey() { return Keys.hmacShaKeyFor(jwtProperties.getSecretKey().getBytes()); }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http){
        JwtTokenValidationFilter jwtTokenValidationFilter = new JwtTokenValidationFilter(secretKey(), jwtProperties);
        return http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(Customizer.withDefaults())
            .sessionManagement( session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests( auth ->
                auth.requestMatchers(🔒 "/api/v1/auth/**").permitAll()
                    .anyRequest().authenticated()
            )
            .addFilterBefore(jwtTokenValidationFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

3.1.3. WebConfiguration

```
@Configuration
public class WebConfiguration {

    @Bean
    public PasswordEncoder passwordEncoder () { return new BCryptPasswordEncoder( strength: 12); }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration authenticationConfiguration) {
        return authenticationConfiguration.getAuthenticationManager();
    }
}
```

3.1.4. JwtProperties

```
package com.michael.a3nguyenvanan18d04.jwt;
```

> import ...

@Component 6 usages
@ConfigurationProperties(prefix = "application.jwt")
@NoArgsConstructor
@AllArgsConstructor
@Slf4j
@Getter
@Setter

```
public class JwtProperties {  
    private String tokenExpirationInDays;  
    private Integer tokenExpirationInHours;  
    private String tokenPrefix;  
    private String headerString;  
}
```

© com.michael.a3nguyenvanan18d04.jwt.JwtProperties
private String headerString
A3NguyenVanAn18D04

3.1.5. JwtTokenProvider

```
20 public class JwtTokenProvider {
21
22     private final SecretKey secretKey; 2 usages
23     private final JwtProperties jwtProperties; 5 usages
24
25     public JwtTokenProvider(SecretKey secretKey, JwtProperties jwtProperties) {
26         this.secretKey = secretKey;
27         this.jwtProperties = jwtProperties;
28     }
29
30
31     /**
32      * Generate JWT token từ Authentication object
33      *
34      * @param authentication Authentication object chứa User Information
35      * @return JWT token string ( Không có Bearer " prefix )
36      */
37
38     public String generateToken(Authentication authentication) { 1 usage
39         try {
40
41             long days = jwtProperties.getTokenExpirationAfterDays() != null ? jwtProperties.getTokenExpirationAfterDays() : 7;
42             long expMillis = System.currentTimeMillis() + days * 24L * 60L * 60L * 1000L;
43             String token = Jwts.builder()
44                 .subject(authentication.getName())
45                 .claim( "s: " "authorities",
46                     authentication.getAuthorities().stream().map(GrantedAuthority::getAuthority).toList()
47                 )
48                 .issuedAt(new Date())
49                 .expiration(new Date(expMillis))
50                 .signWith(secretKey)
51                 .compact();
52             log.info("Create a jwt token {}", authentication.getAuthorities());
53             return token;
54         } catch (Exception e) {
55             log.error("Error while creating a jwt token: {}", e.getMessage());
56         }
57     }
```

```

26 public class JwtTokenProvider {
27
28     public String generateToken(Authentication authentication) { 1 usage
29         try {
30
31             long days = jwtProperties.getTokenExpirationAfterDays() != null ? jwtProperties.getTokenExpirationAfterDays() : 7;
32             long expMillis = System.currentTimeMillis() + days * 24L * 60L * 60L * 1000L;
33             String token = Jwts.builder()
34                 .subject(authentication.getName())
35                 .claim("authorities",
36                     authentication.getAuthorities().stream()
37                         .map(GrantedAuthority::getAuthority)
38                         .toList())
39                 .issuedAt(new Date())
40                 .expiration(new Date(expMillis))
41                 .signWith(secretKey)
42                 .compact();
43             log.info("Create a jwt token {}", authentication.getAuthorities());
44             return token;
45         } catch (Exception e) {
46             log.error("Error while creating a jwt token: {}", e.getMessage());
47             return null;
48         }
49     }
50
51     public String generateTokenWithPrefix (Authentication authentication) { 1 usage
52         log.info("Create a jwt token successfully with prefix {}", jwtProperties.getTokenPrefix());
53         return jwtProperties.getTokenPrefix() + " " + generateToken(authentication);
54     }
55
56 }
57
58

```

3.1.6. JwtTokenValidationFilter

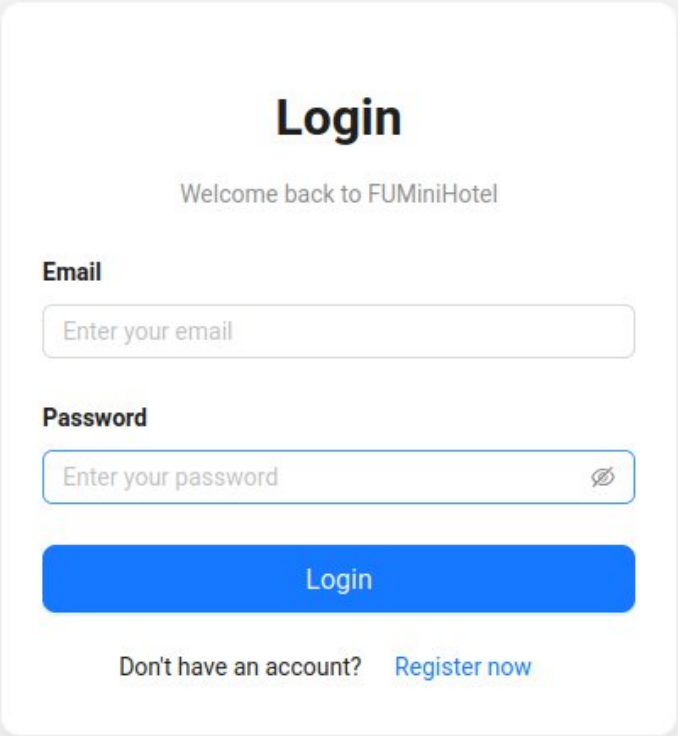
```
1 package com.michael.a3nguyenvanan18d04.jwts;
2
3 > import ...
20
21 > /** JWT Token Validation Filter - Xác thực JWT token cho các request tiếp theo ...*/
50 @Slf4j 3 usages
51 ✨ public class JwtTokenValidationFilter extends OncePerRequestFilter {
52     private final SecretKey secretKey; 2 usages
53     private final String tokenPrefix; 3 usages
54
55 @ public JwtTokenValidationFilter(SecretKey secretKey, JwtProperties jwtProperties) { 1 usage
56     this.secretKey = secretKey;
57     this.tokenPrefix = jwtProperties.getTokenPrefix();
58 }
59
60 > /** Thực hiện filter logic để xác thực JWT token. ...*/
80 @Override no usages
81 📌 protected void doFilterInternal(@NonNull HttpServletRequest request,
82                                     @NonNull HttpServletResponse response,
83                                     @NonNull FilterChain filterChain) throws
84                                     IOException {
85     try {
86         String path = request.getServletPath();
87
88         if (path.startsWith("/api/v1/auth/login") || path.startsWith("/api/v1/auth/register"))
89             filterChain.doFilter(request, response);
90         return;
91     }
92
93     String authorizationHeader = request.getHeader("Authorization");
94     if(authorizationHeader == null || !authorizationHeader.startsWith(tokenPrefix)) {
95         filterChain.doFilter(request, response);
96         return;
97     }
98
99     String token = authorizationHeader.replace(tokenPrefix, "").trim();
100
101     Claims claims = Jwts.parser().setSigningKey(secretKey)
102                     .parseClaimsJws(token).getBody();
```

```

51 public class JwtTokenValidationFilter extends OncePerRequestFilter {
81     protected void doFilterInternal(@NonNull HttpServletRequest request,
103         .build() JwtParser
104         .parseSignedClaims(token) Jws<Claims>
105         .getPayload();
106
107     String username = claims.getSubject();
108     Object authoritiesObj = claims.get("authorities");
109
110     List<SimpleGrantedAuthority> grantedAuthorities;
111     if (authoritiesObj instanceof List<?> authList) {
112         grantedAuthorities = authList.stream() Stream<capture of ?>
113             .map(Object::toString) Stream<String>
114             .map(SimpleGrantedAuthority::new) Stream<SimpleGrantedA
115             .toList();
116     } else {
117         grantedAuthorities = List.of();
118     }
119
120     log.debug("User: {}, Authorities: {}", username, grantedAuthorities);
121     Authentication authentication = new UsernamePasswordAuthenticationToken(username, crede
122     SecurityContextHolder.getContext().setAuthentication(authentication);
123     filterChain.doFilter(request, response);
124
125     } catch (JwtException e) {
126         log.error("JWT validation failed: {}", e.getMessage());
127         response.setStatus(HttpStatus.SC_UNAUTHORIZED);
128         response.setContentType("application/json");
129         response.getWriter().write(s: "{\"error\": \"Invalid or expired token\"}");
130     } catch (Exception e) {
131         log.error("Unexpected error in JWT filter: {}", e.getMessage());
132         response.setStatus(HttpStatus.SC_INTERNAL_SERVER_ERROR);
133         response.setContentType("application/json");
134         response.getWriter().write(s: "{\"error\": \"Internal server error\"}");
135     }
136
137 }
138 }
139

```

3.1.7. Result



The image shows a login form for 'FUMiniHotel'. The form is centered on a light gray background. It has a white background with rounded corners and a subtle drop shadow. At the top, the word 'Login' is written in a large, bold, black font. Below it, the text 'Welcome back to FUMiniHotel' is displayed in a smaller, gray font. The form contains two input fields: one for 'Email' with the placeholder text 'Enter your email', and one for 'Password' with the placeholder text 'Enter your password' and a toggle icon (an eye with a slash) on the right. Below the password field is a blue button with the text 'Login' in white. At the bottom of the form, there is a link that says 'Don't have an account? Register now'.

Login

Welcome back to FUMiniHotel

Email

Enter your email

Password

Enter your password 

Login

Don't have an account? [Register now](#)

Register Account

Join FUMiniHotel to book your rooms

Full Name

Email

Password

Confirm Password

Phone Number

ID Card Number

Date of Birth

Gender
☒ Male ☐ Female ☐ Other

Register

Already have an account? [Login here](#)

3.2. Customer Management

3.2.1. Customer

```

public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long customerID;

    private String customerFullName;

    private String telephone;

    private String emailAddress;

    private LocalDate customerBirthday;

    private CustomerStatus customerStatus;

    private String password;

    private Role roles;

    @OneToMany(mappedBy = "customer", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    @JsonIgnoreProperties("customer")
    private List<BookingReservation> bookingReservation = new ArrayList<>();

    public void addBookingReservation(BookingReservation bookingReservation) { 1 usage
        this.bookingReservation.add(bookingReservation);
        bookingReservation.setCustomer(this);
    }

    public void removeBookingReservation(BookingReservation bookingReservation) { no usages
        this.bookingReservation.remove(bookingReservation);
        bookingReservation.setCustomer(this);
    }
}

```

3.2.2. CustomerRepository

```

package com.michael.a3nguyenvanan18d04.repository;

import ...

@Repository 3 usages
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    Customer findByEmailAddress(String emailAddress); 2 usages
    Customer findByTelephone(String telephone); no usages
}

```


3.2.3. CustomerServiceImpl

```
public class CustomerServiceImpl implements CustomerService, UserDetailsService {
    private final CustomerRepository customerRepository; 9 usages
    > public CustomerServiceImpl(CustomerRepository customerRepository) { this.customerRepository = customerRepository; }
    @Override no usages
    public @NonNull UserDetails loadUserByUsername(@NonNull String username) throws UsernameNotFoundException {
        Customer customer = customerRepository.findByEmailAddress(username);
        if (customer == null) {
            throw new UsernameNotFoundException("User not found");
        }
        return User.builder().username(customer.getEmailAddress()).password(customer.getPassword()).roles(customer.getRoles()
    }
    @Override 2 usages
    > public List<Customer> getCustomers() { return customerRepository.findAll(); }
    @Override 2 usages
    > public Customer getCustomerById(Long id) { return customerRepository.findById(id).orElse( other: null); }
    @Override 8 usages
    > public Customer createCustomer(Customer customer) { return customerRepository.save(customer); }
    @Override 1 usage
    public Customer updateCustomer(Long id, Customer customer) {
        if (customer == null) {
            return null;
        }
        Customer existingCustomer = customerRepository.getReferenceById(id);
        existingCustomer.setCustomerFullName(customer.getCustomerFullName());
        existingCustomer.setTelephone(customer.getTelephone());
        existingCustomer.setEmailAddress(customer.getEmailAddress());
        existingCustomer.setCustomerBirthday(customer.getCustomerBirthday());
        existingCustomer.setCustomerStatus(customer.getCustomerStatus());
        existingCustomer.setPassword(customer.getPassword());
        return customerRepository.save(existingCustomer);
    }
    @Override 1 usage
    > public void deleteCustomer(Long id) { customerRepository.deleteById(id); }
    @Override 2 usages
    > public Customer getCustomerByEmail(String email) { return customerRepository.findByEmailAddress(email); }
}
```


3.2.4. CustomerController

```
package com.michael.a3nguyenvan18d04.controllers;

import ...

@RestController
@RequestMapping("/api/v1/customers")

public class CustomerController {
    private final CustomerService customerService; 7 usages
    public CustomerController(CustomerService customerService) { this.customerService = customerService; }

    @GetMapping("/")
    public ResponseEntity<Object> getAllCustomers() { return ResponseEntity.ok(customerService.getCustomers()); }

    @PostMapping("/")
    public ResponseEntity<Object> addCustomer(@RequestBody Customer customer) {
        return ResponseEntity.ok(customerService.createCustomer(customer));
    }

    @GetMapping("/{id}")
    public ResponseEntity<Object> getCustomerById(@PathVariable Long id) {
        return ResponseEntity.ok(customerService.getCustomerById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Object> updateCustomer(@PathVariable Long id, @RequestBody Customer customer) {
        if(customerService.updateCustomer(id, customer) != null) {
            return ResponseEntity.ok(customerService.getCustomers());
        }
        return ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Object> deleteCustomer(@PathVariable Long id) {
        customerService.deleteCustomer(id);
        return ResponseEntity.ok(body: "Customer deleted");
    }
}
```

3.2.5. Result

Customer Management + Add Customer

ID	Full Name	Email	Phone	Status	Action
#1	Admin User	admin@codebase.com	0901234567	INACTIVE	View Edit Deactivate
#2	Manager User	manager@codebase.com	0901234568	INACTIVE	View Edit Deactivate
#3	User One	user1@codebase.com	0901234569	INACTIVE	View Edit Deactivate
#4	User Two	user2@codebase.com	0901234570	INACTIVE	View Edit Deactivate
#5	Locked User	locked@codebase.com	0901234571	INACTIVE	View Edit Deactivate
#6	Disabled User	disabled@codebase.com	0901234572	INACTIVE	View Edit Deactivate

< 1 >

Customer Management

+ Add Customer

ID	Full Name	Email	Phone	Status	Action
#1	Admin User	admin@codebase.com		ACTIVE	View Edit Deactivate
#2	Manager User	manager@codebase.com		ACTIVE	View Edit Deactivate
#3	User One	user1@codebase.com		ACTIVE	View Edit Deactivate
#4	User Two	user2@codebase.com		ACTIVE	View Edit Deactivate
#5	Locked User	locked@codebase.com		LOCKED	View Edit Deactivate
#6	Disabled User	disabled@codebase.com		DISABLED	View Edit Deactivate

Edit Customer

* Full Name

Admin User

* Email

admin@codebase.com

* Telephone

0901234567

Birthday

Select date

* Status

ACTIVE

Cancel

Update Customer

U

User Two

CUSTOMER

EMAIL

user2@codebase.com

PHONE

0901234570

Account Settings

Full Name

User Two

Email Address (Read-only) (optional)

user2@codebase.com

Phone Number

0901234570

Date of Birth (optional)

01/04/1996

Save Changes

3.3. Quản lý phòng (RoomInformationController)

3.3.1. RoomInformation

```
package com.michael.a3nguyenvan18d04.entites;
> import ...
@Entity
@Table(name = "RoomInformation")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class RoomInformation {
    @Id
    @GeneratedValue(strategy = jakarta.persistence.GenerationType.IDENTITY)
    private Long roomID;
    private String roomNumber;
    private String roomDetailDescription;

    private int roomMaxCapacity;

    private RoomStatus roomStatus;

    private double roomPricePerDay;

    @ManyToOne
    @JoinColumn(name = "roomTypeID")
    @JsonIgnoreProperties("roomInformation")
    private RoomType roomType;

    @OneToMany(mappedBy = "roomInformation", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    @JsonIgnoreProperties("roomInformation")
    private List<BookingDetail> bookingDetails = new ArrayList<>();

    public void addBookingDetail(BookingDetail bookingDetail) { 1 usage
        this.bookingDetails.add(bookingDetail);
        bookingDetail.setRoomInformation(this);
    }
    public void removeBookingDetail(BookingDetail bookingDetail) { no usages
        this.bookingDetails.remove(bookingDetail);
        bookingDetail.setRoomInformation(null);
    }
}
```

3.3.2. RoomType

```
package com.michael.a3nguyenvanan18d04.entites;

import ...
@Entity
@Table
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class RoomType {

    @Id
    @GeneratedValue(strategy= GenerationType.IDENTITY)
    private Long roomTypeID;
    private String roomTypeName;

    private String roomTypeDescription;

    private String roomTypeNote;

    @OneToMany(mappedBy = "roomType", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)
    @JsonIgnoreProperties("roomType")
    private List<RoomInformation> roomInformation = new ArrayList<>();

    public void addRoomInformation(RoomInformation roomInformation) { 6 usages
        if (roomInformation == null) return;
        if (this.roomInformation == null) this.roomInformation = new ArrayList<>();
        if (!this.roomInformation.contains(roomInformation)) {
            this.roomInformation.add(roomInformation);
            roomInformation.setRoomType(this);
        }
    }

    public void removeRoomInformation(RoomInformation roomInformation) { no usages
        if (roomInformation == null || this.roomInformation == null) return;
        if (this.roomInformation.remove(roomInformation)) {
            roomInformation.setRoomType(null);
        }
    }
}
```

3.3.3. RoomInformationRepository

```
package com.michael.a3nguyenvanan18d04.repository;

import ...

@Repository 7 usages
public interface RoomInformationRepository extends JpaRepository<RoomInformation, Long> {
}
```

3.3.4. RoomInformationService

```
1 package com.michael.a3nguyenanani804.services.impl;
2
3 > import ..
4
14
15 @Service
16 public class BookingReservationServiceImpl implements BookingReservationService {
17     private final BookingReservationRepository bookingReservationRepository; 6 usages
18     private final CustomerService customerService; 3 usages
19     private final BookingDetailsService bookingDetailsService; 2 usages
20
21     public BookingReservationServiceImpl(BookingReservationRepository bookingReservationRepository, CustomerServiceImpl customerService, BookingDetailsService bookingDetailsService) {
22         this.bookingReservationRepository = bookingReservationRepository;
23         this.customerService = customerService;
24         this.bookingDetailsService = bookingDetailsService;
25     }
26
27     @Override 1 usage
28     public List<BookingReservation> getBookingReservations(Authentication authentication) {
29         boolean isStaff = authentication.getAuthorities().stream().map(a -> a.getAuthority()) Stream<capture of extends GrantedAuthority>
30             .map(a -> a.getAuthority()) Stream<String>
31             .anyMatch(role -> role.equals("ROLE_STAFF") || role.equals("ROLE_ADMIN") || role.equals("ROLE_MANAGER"));
32         if (isStaff) {
33             return bookingReservationRepository.findAll();
34         }
35         String principalName = authentication.getName();
36         Customer customer = customerService.getCustomerByEmail(principalName);
37         return customer.getBookingReservation();
38     }
39     @Override 1 usage
40     public BookingReservation getBookingReservationById(Long id) {
41         return bookingReservationRepository.getReferenceById(id);
42     }
```

```
    @Override 1 usage
    public BookingReservation getBookingReservationById(Long id) {
        return bookingReservationRepository.getReferenceById(id);
    }
    @Override 1 usage
    public BookingReservation createBookingReservation(BookingRequestDTO bookingRequest) {
        BookingReservation bookings= BookingReservation.builder().bookingDate(bookingRequest.getBookingDate()).bookingStatus(bookingRequest.getBookingStatus()).totalPT
        List<BookingDetail> bookingDetails =
            bookingRequest.getBookingDetails() List<BookingDetailRequestDTO>
            .stream() Stream<BookingDetailRequestDTO>
            .map(bookingDetailsService::createBookingDetail).toList();
        for (BookingDetail detail : bookingDetails) {
            bookings.addBookingDetails(detail);
        }
        Customer customer = customerService.getCustomerById(bookingRequest.getCustomerID());
        customer.addBookingReservation(bookings);
        return bookingReservationRepository.save(bookings);
    }
    @Override 1 usage
    public BookingReservation updateBookingReservation(Long id, BookingReservation bookingReservation) {
        return bookingReservationRepository.save(bookingReservation);
    }
    @Override no usages
    public void deleteBookingReservation(Long id) {
        bookingReservationRepository.deleteById(id);
    }
}
```

Invalid VCS root mapping
The directory
~/code/SBA301-Spring-Boot-Application-With-React-JS/

3.3.5. RoomInformationController

```
package com.michael.a3nguyenvanan18d04.controllers;

import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;

@RestController
@RequestMapping("/api/v1/rooms")
@Slf4j
public class RoomInformationController {

    private final RoomInformationService roomInformationService;

    public RoomInformationController(RoomInformationService roomInformationService) {
        this.roomInformationService = roomInformationService;
    }

    @GetMapping
    public Object getRoomInformation() { return roomInformationService.getRoomInformationServices(); }

    @PutMapping("/{id}")
    public Object updateRoomInformation(@PathVariable Long id, @RequestBody UpdateRoomRequest updateRoomRequest) {
        return roomInformationService.updateRoomInformation(id, updateRoomRequest);
    }

    @PostMapping
    public Object createRoomInformation(@RequestBody UpdateRoomRequest updateRoomRequest) {
        return roomInformationService.createRoomInformation(updateRoomRequest);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Object> deleteRoomInformation(@PathVariable Long id) {
        roomInformationService.deleteRoomInformation(id);
        return ResponseEntity.ok().build();
    }
}
```

Staff Administration Panel

Admin User
STAFF

Logout

Rooms Management

+ Add New Room

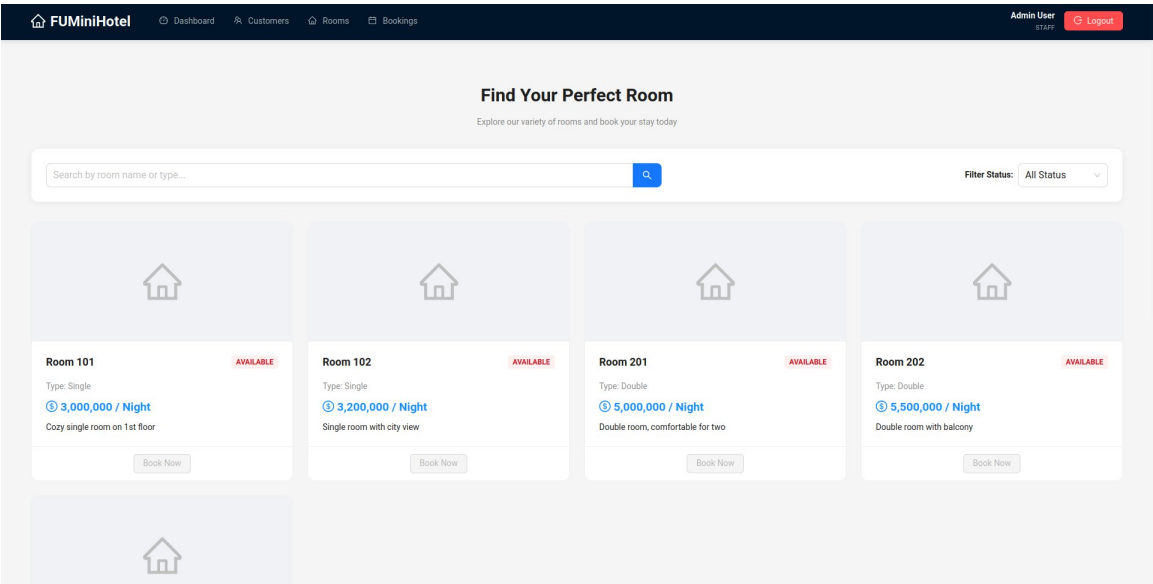
Search by room name or type...

Filter Status: All Status

ID	Room Number	Type	Price/Night	Status	Action
#	101	Single	3,000,000 VND	AVAILABLE	Edit Delete
#	102	Single	3,200,000 VND	AVAILABLE	Edit Delete
#	201	Double	5,000,000 VND	AVAILABLE	Edit Delete
#	202	Double	5,500,000 VND	AVAILABLE	Edit Delete
#	301	Deluxe	120 VND	AVAILABLE	Edit Delete

< 1 >

3.3.6. Result



3.4. Quản lý Đặt phòng (BookingReservationController)

3.4.1. BookingReservation

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class BookingReservation {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long bookingReservationID;

    private LocalDateTime bookingDate;

    private double totalPrice;

    private BookingStatus bookingStatus;

    @ManyToOne
    @JoinColumn(name = "customerID")
    @JsonIgnoreProperties("bookingReservation")
    private Customer customer;

    @OneToMany(mappedBy = "bookingReservation", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    @JsonIgnoreProperties("bookingReservation")
    private List<BookingDetail> bookingDetails = new ArrayList<>();

    public void addBookingDetails(BookingDetail detail) { 1 usage
        if (bookingDetails == null) {
            bookingDetails = new ArrayList<>();
        }
        bookingDetails.add(detail);
        detail.setBookingReservation(this); // set back-reference
    }

    public void removeBookingDetails(BookingDetail detail) { no usages
        if (bookingDetails != null) {
            bookingDetails.remove(detail);
            detail.setBookingReservation(null);
        }
    }
}
```


3.4.2. BookingReservationRepository

```
package com.michael.a3nguyenvanan18d04.repository;

import ...

@Repository 3 usages
public interface BookingReservationRepository extends JpaRepository<BookingReservation, Long> {

}
```

3.4.3. BookingReservationServiceImpl

```
package com.michael.a3nguyenvanan18d04.services.impl;

import ...

@Service
public class BookingReservationServiceImpl implements BookingReservationService {
    private final BookingReservationRepository bookingReservationRepository;
    private final CustomerService customerService;
    private final BookingDetailService bookingDetailService;

    public BookingReservationServiceImpl(BookingReservationRepository bookingReservationRepository, CustomerServiceImpl customerService, BookingDetailService bookingDetailService) {
        this.bookingReservationRepository = bookingReservationRepository;
        this.customerService = customerService;
        this.bookingDetailService = bookingDetailService;
    }

    @Override 1 usage
    public List<BookingReservation> getBookingReservations(Authentication authentication) {
        boolean isStaff = authentication.getAuthorities().stream().stream().captureOf extends GrantedAuthority>
            .map(a -> a.getAuthority()) Stream<String>
            .anyMatch(role -> role.equals("ROLE_STAFF") || role.equals("ROLE_ADMIN") || role.equals("ROLE_MANAGER"));

        if (isStaff) {
            return bookingReservationRepository.findAll();
        }
        String principalName = authentication.getName();
        Customer customer = customerService.getCustomerByEmail(principalName);
        return customer.getBookingReservations();
    }

    @Override 1 usage
    public BookingReservation getBookingReservationById(Long id) {
        return bookingReservationRepository.getReferenceById(id);
    }

    @Override 1 usage
    public BookingReservation createBookingReservation(BookingRequestDTO bookingRequest) {
        BookingReservation bookings = BookingReservation.builder().bookingDate(bookingRequest.getBookingDate()).bookingStatus(bookingRequest.getBookingStatus()).totalPrice(bookingRequest.getTotalPrice()).build();
        List<BookingDetail> bookingDetails =
            bookingRequest.getBookingDetails().stream().stream().captureOf extends BookingDetailRequestDTO>
                .stream() Stream<BookingDetailRequestDTO>
                .map(bookingDetailService::createBookingDetail).toList();
        for (BookingDetail detail : bookingDetails) {
            bookings.addBookingDetails(detail);
        }
        Customer customer = customerService.getCustomerById(bookingRequest.getCustomerId());
        customer.addBookingReservation(bookings);
        return bookingReservationRepository.save(bookings);
    }

    @Override 1 usage
    public BookingReservation updateBookingReservation(Long id, BookingReservation bookingReservation) {
        return bookingReservationRepository.save(bookingReservation);
    }

    @Override no usages
    public void deleteBookingReservation(Long id) {
        bookingReservationRepository.deleteById(id);
    }
}
```

3.4.4. BookingReservationController

```
package com.michael.a3nguyenvanan18d04.controllers;

import com.michael.a3nguyenvanan18d04.dtos.booking.request.BookingRequestDTO;
import com.michael.a3nguyenvanan18d04.dtos.booking.request.BookingStatusUpdateDTO;
import com.michael.a3nguyenvanan18d04.entities.BookingReservation;
import com.michael.a3nguyenvanan18d04.services.interfaces.BookingReservationService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/bookings")
public class BookingReservationController {
    private final BookingReservationService bookingReservationService;

    public BookingReservationController(BookingReservationService bookingReservationService){
        this.bookingReservationService = bookingReservationService;
    }

    @GetMapping("/{customer}/{customerID}")
    @PreAuthorize("hasAnyRole('STAFF','CUSTOMER')")
    public ResponseEntity<Object> getBookings(@PathVariable(required = false) Long customerID, Authentication authentication) {
        return ResponseEntity.ok(
            bookingReservationService.getBookingReservations(authentication)
        );
    }

    @PostMapping
    public ResponseEntity<Object> createBooking(@RequestBody BookingRequestDTO bookingRequestDTO ) {
        return ResponseEntity.ok(
            bookingReservationService.createBookingReservation(bookingRequestDTO)
        );
    }

    @PutMapping("/{id}/{id}/status")
    public ResponseEntity<Object> updateBookingStatus(@PathVariable Long id, @RequestBody BookingStatusUpdateDTO bookingStatusUpdateDTO) {
        try {
            BookingReservation bookings = bookingReservationService.getBookingReservationById(id);
            bookings.setBookingStatus(bookingStatusUpdateDTO.getStatus());
            bookingReservationService.updateBookingReservation(id, bookings);
        } catch (Exception e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
        return ResponseEntity.ok( body: "Booking status updated");
    }
}
```

3.4.5. Result

Book Your Stay

 Select Rooms —————  Set Dates —————  Review & Confirm

Choose from available rooms

Room 101

Single

3,000,000 VND / Night

Room 102

Single

3,200,000 VND / Night

Room 201

Double

5,000,000 VND / Night

Room 202

Double

5,500,000 VND / Night

Room 301

Deluxe

120 VND / Night

Previous

Next

Book Your Stay

 Select Rooms —————  Set Dates —————  Review & Confirm

Set dates for each room

Room 101 - Single

3,000,000 VND / night

Check-in:

Check-in Date 

Check-out:

Check-out Date 

 Please select check-in and check-out dates

Room 102 - Single

3,200,000 VND / night

Check-in:

Check-in Date 

Check-out:

Check-out Date 

 Please select check-in and check-out dates

Previous

Next

Review your booking

Customer & Pricing

Guest Name	User Two
Total Amount	54,200,000 VND

Room Details

Room 101

Dates: 04/03/2026 - 05/03/2026
Type: Single

3,000,000 VND
1 nights x 3,000,000

Room 102

Dates: 02/03/2026 - 18/03/2026
Type: Single

51,200,000 VND
16 nights x 3,200,000



Important Information

By clicking confirm, you agree to our booking terms and conditions. Payment will be handled at the hotel.

[Previous](#)

[Confirm Reservation](#)

My Booking History

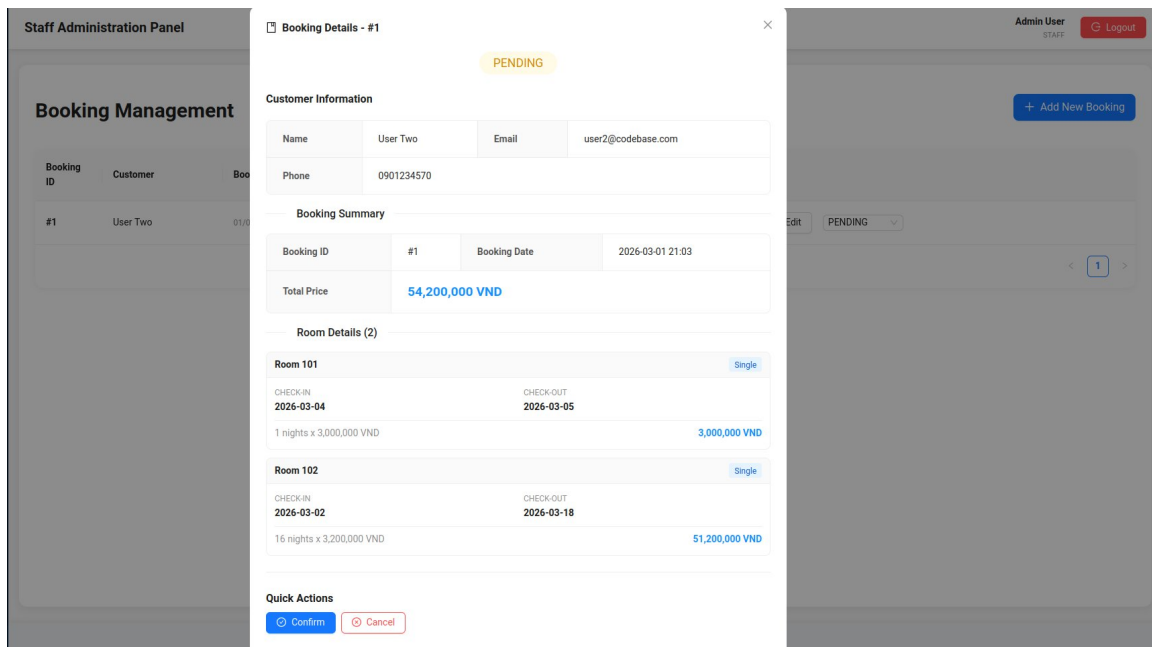
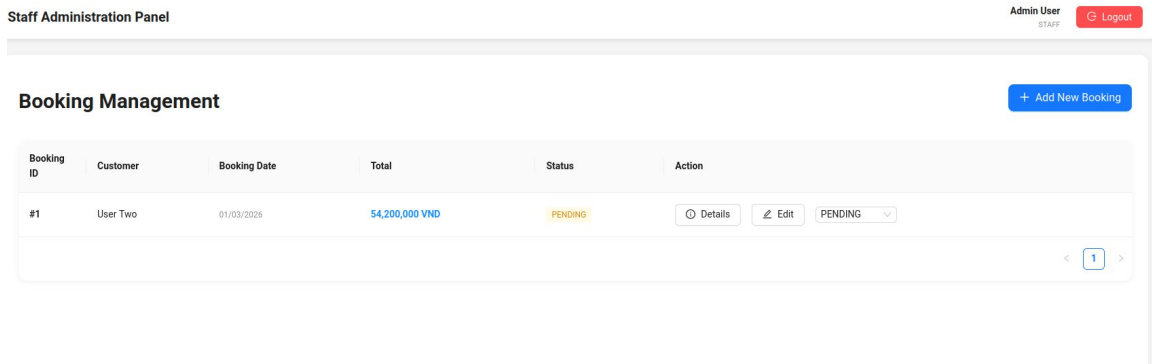
Review your previous and upcoming stays at FUMiniHotel

Booking ID	Booking Date	Total	Status	Action
#1	01/03/2026	54,200,000 VND	PENDING	Details

< 1 >

[Quick Links](#)

[Contact Info](#)



4. FRONTEND: DỊCH VỤ & HIỂN THỊ DỮ LIỆU

4.1. Cấu hình Core & Bảo mật (Core Configuration)

4.1.1. axiosInstance.js (Kết nối API)

- Chức năng: Khởi tạo thực thể Axios để giao tiếp với Backend.
- Chi tiết: Cấu hình baseURL trỏ tới Server Spring Boot và thiết lập Content-Type: application/json.

```

1 > import ...
2
3 const axiosInstance : AxiosInstance = axios.create({
4   baseURL: API_BASE_URL,
5   headers: {
6     'Content-Type': 'application/json',
7   },
8 });
9
10 // Request Interceptor
11 axiosInstance.interceptors.request.use( & Michaelht271
12   onFulfilled: (config) => {
13     // Don't add token for login/register requests
14     const isPublicAuth = config.url.includes(API_ENDPOINTS.AUTH.LOGIN) || config.url.includes(API_ENDPOINTS.AUTH.REGISTER);
15
16     if (!isPublicAuth) {
17       const token : string = localStorage.getItem(key: 'token');
18       if (token) {
19         config.headers.Authorization = token.startsWith('Bearer ') ? token : `Bearer ${token}`;
20       }
21     }
22     return config;
23   },
24   onRejected: (error) => {
25     return Promise.reject(error);
26   }
27 );

```

```

// Response Interceptor
axiosInstance.interceptors.response.use( & Michaelht271
  onFulfilled: (response) => response,
  onRejected: (error) => {
    // Log the error in console for debugging (helpful for 403)
    console.error('API Error [{error.response?.status}]:', error.response?.data || error.message);

    if (error.response?.status === 401) {
      localStorage.removeItem(key: 'token');
      localStorage.removeItem(key: 'user');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default axiosInstance; Show usages & Michaelht271

```

4.1.2. apiClient.js (Interceptors & JWT)

- Chức năng: Tự động đính kèm Token vào mọi yêu cầu gửi đi.
- Logic:
 - Request Interceptor: Kiểm tra localStorage, nếu có Token sẽ thêm vào header
Authorization: Bearer <Token>.
 - Response Interceptor: Bắt lỗi 401 (Unauthorized) để tự động đăng xuất người dùng nếu Token hết hạn.

```

1  import axiosInstance from './axiosInstance';  Michaelht271, Today · finish as3
2
3  const apiClient: {...} = {
4    get: async (url, config: {} = {}) => {
5      const response: AxiosResponse<any> = await axiosInstance.get(url, config);
6      return response.data;
7    },
8    post: async (url, data, config: {} = {}) => {
9      const response: AxiosResponse<any> = await axiosInstance.post(url, data, config);
10     return response.data;
11   },
12   put: async (url, data, config: {} = {}) => {
13     const response: AxiosResponse<any> = await axiosInstance.put(url, data, config);
14     return response.data;
15   },
16   delete: async (url, config: {} = {}) => {
17     const response: AxiosResponse<any> = await axiosInstance.delete(url, config);
18     return response.data;
19   },
20 };
21
22 export default apiClient;  Show usages  Michaelht271
23

```

4.1.3. AuthContext.jsx (Quản lý trạng thái toàn cục)

- Chức năng: Lưu trữ thông tin người dùng và trạng thái đăng nhập xuyên suốt ứng dụng.
- Logic: Sử dụng useContext và useState để cung cấp các hàm login, logout cho các component con.

```

import React, {createContext, useState, useEffect, useContext} from 'react';
import {decodeToken, isTokenExpired} from "../jwtUtils.js"; // Michaelht271, Today • finish as3
const AuthContext : Context<null> = createContext<default>(null);

const AuthProvider = ({ children }) => { Show usages ⚠ Michaelht271
  const [user, setUser] = useState<initialState>(null);
  const [token, setToken] = useState<initialState>(null);
  const [isAuthenticated : boolean, setIsAuthenticated] = useState<initialState>(false);
  const [loading : boolean, setLoading] = useState<initialState>(true);

  useEffect<effect>(() => {
    // Check localStorage on load
    const storedToken : string = localStorage.getItem<key>('token');
    const storedUserStr : string = localStorage.getItem<key>('user');

    if (storedToken && !isTokenExpired(storedToken)) {
      setToken(storedToken);
      setIsAuthenticated<value>(true);

      if (storedUserStr) {
        try {
          const storedUser = JSON.parse(storedUserStr);
          // Handle 'roles' as a collection from Java if needed during rehydration
          let currentRole = storedUser.role;
          if (storedUser.roles && Array.isArray(storedUser.roles)) {
            const roleObj = storedUser.roles.find(r => (typeof r === 'string' ? r : r.authority)?.startsWith('ROLE_'));
            const roleString : string | any = (typeof roleObj === 'string' ? roleObj : roleObj?.authority);
            if (roleString) currentRole = roleString.replace<searchValue>('ROLE_', <replaceValue> '');
          }
        }
      }
    }
  });

  const id = storedUser.customerID || storedUser.customerId || storedUser.id;
  const normalizedUser = {
    ...storedUser,
    role: currentRole || 'CUSTOMER',
    customerID: id,
    customerId: id,
    id: id,
    fullName: storedUser.fullName || storedUser.customerFullName,
    customerFullName: storedUser.fullName || storedUser.customerFullName,
  };
  setUser(normalizedUser);
} catch (e) {
  console.error("Failed to parse stored user", e);
}
} else {
  // Fallback: decode token
  const decoded : JwtPayload = decodeToken(storedToken);
  if (decoded) {
    const authorities = decoded.authorities || [];
    const rawRole : string = authorities.find(a : T => a.startsWith('ROLE_')) || 'CUSTOMER';
    const normalizedRole : string = rawRole.replace<searchValue>('ROLE_', <replaceValue> '');

    const id = decoded.customerID || decoded.customerId || decoded.id;
    setUser<value>({
      email: decoded.sub,
      emailAddress: decoded.sub,
    });
  }
}

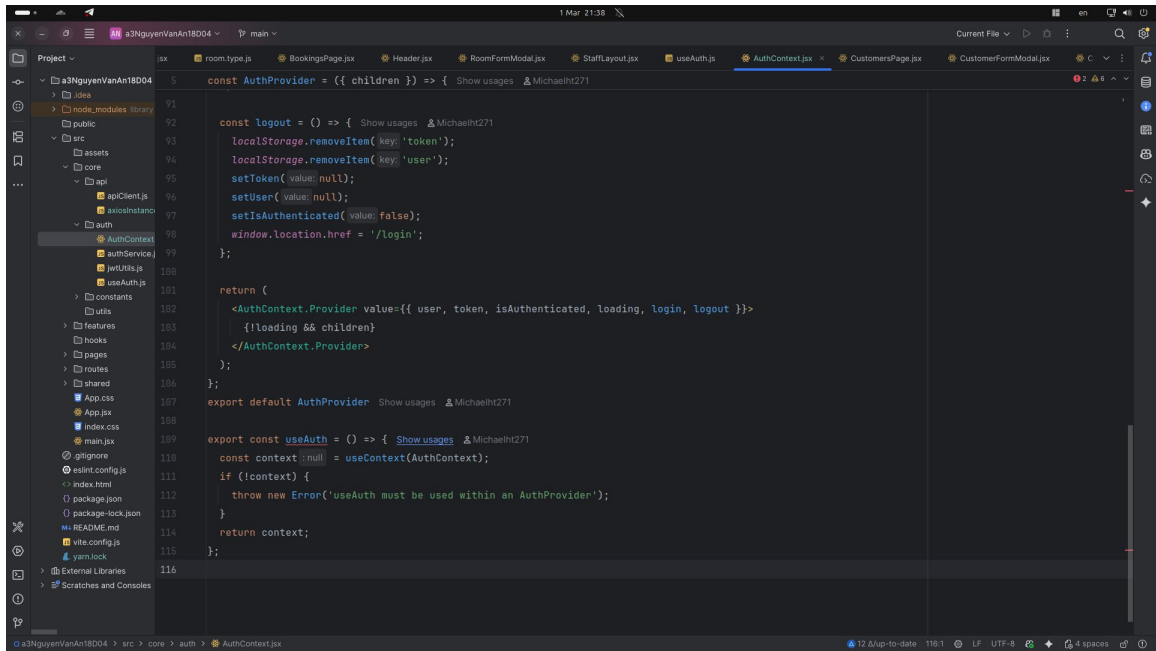
```

```

const id = storedUser.customerID || storedUser.customerId || storedUser.id;
const normalizedUser = {
  ...storedUser,
  role: currentRole || 'CUSTOMER',
  customerID: id,
  customerId: id,
  id: id,
  fullName: storedUser.fullName || storedUser.customerFullName,
  customerFullName: storedUser.fullName || storedUser.customerFullName,
};
setUser(normalizedUser);
} catch (e) {
  console.error("Failed to parse stored user", e);
}
} else {
  // Fallback: decode token
  const decoded : JwtPayload = decodeToken(storedToken);
  if (decoded) {
    const authorities = decoded.authorities || [];
    const rawRole : string = authorities.find(a : T => a.startsWith('ROLE_')) || 'CUSTOMER';
    const normalizedRole : string = rawRole.replace<searchValue>('ROLE_', <replaceValue> '');

    const id = decoded.customerID || decoded.customerId || decoded.id;
    setUser<value>({
      email: decoded.sub,
      emailAddress: decoded.sub,
    });
  }
}

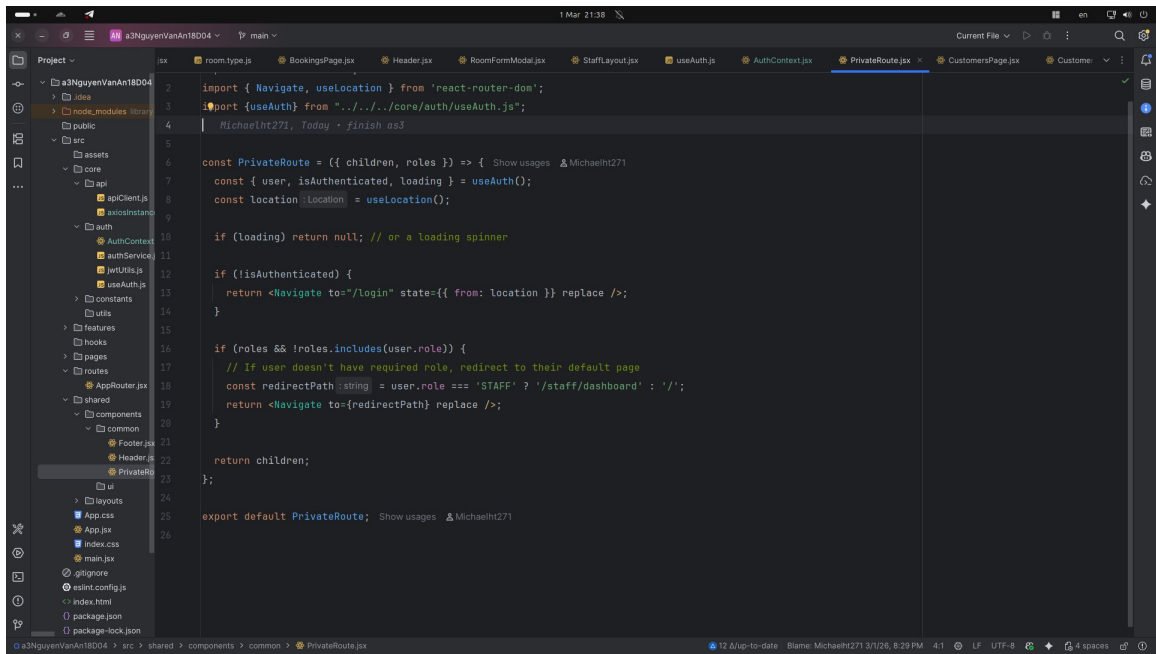
```

```
const AuthProvider = ({ children }) => {  
  const logout = () => {  
    localStorage.removeItem(key: 'token');  
    localStorage.removeItem(key: 'user');  
    setToken(value: null);  
    setUser(value: null);  
    setIsAuthenticated(value: false);  
    window.location.href = '/login';  
  };  
  
  return (  
    <AuthContext.Provider value={{ user, token, isAuthenticated, loading, login, logout }}>  
      {loading && children}  
    </AuthContext.Provider>  
  );  
};  
  
export default AuthProvider;  
  
export const useAuth = () => {  
  const context = useContext(AuthContext);  
  if (!context) {  
    throw new Error('useAuth must be used within an AuthProvider');  
  }  
  return context;  
};
```

4.1.4. PrivateRoute.jsx (Phân quyền truy cập)

- Chức năng: Bảo vệ các tuyến đường (routes) yêu cầu đăng nhập.
- Logic: Kiểm tra Role của người dùng. Nếu khách hàng cố tình truy cập trang của nhân viên (Staff), hệ thống sẽ chuyển hướng về trang /unauthorized.

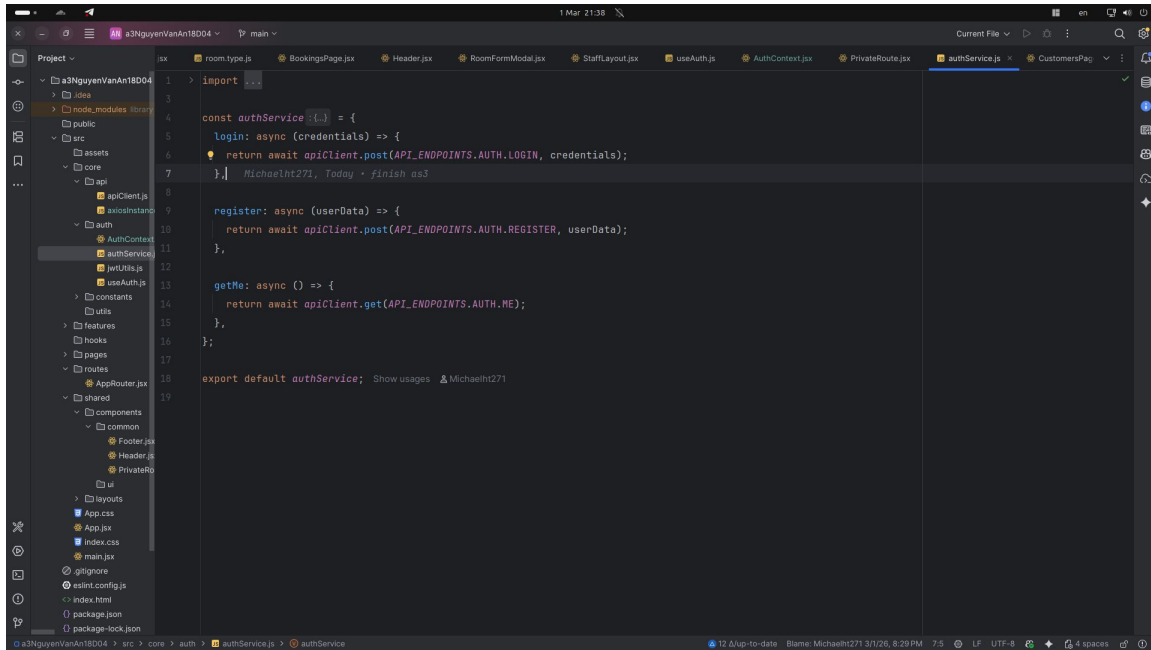


```
import { Navigate, useLocation } from 'react-router-dom';  
import { useAuth } from '../core/auth/useAuth.js';  
  
const PrivateRoute = ({ children, roles }) => {  
  const { user, isAuthenticated, loading } = useAuth();  
  const location = useLocation();  
  
  if (loading) return null; // or a loading spinner  
  
  if (!isAuthenticated) {  
    return <Navigate to="/login" state={{ from: location }} replace />;  
  }  
  
  if (roles && !roles.includes(user.role)) {  
    // If user doesn't have required role, redirect to their default page  
    const redirectPath = user.role === 'STAFF' ? '/staff/dashboard' : '/';  
    return <Navigate to={redirectPath} replace />;  
  }  
  
  return children;  
};  
  
export default PrivateRoute;
```

4.2. Feature: Authentication (Đăng nhập & Đăng ký)

4.2.1. authService.js

- Chức năng: Chứa các hàm gọi API /api/auth/login và /api/auth/register.
- Kết quả: Trả về dữ liệu người dùng và Token cho AuthContext xử lý.



```
1 > import ...
2
3
4 const authService = {
5   login: async (credentials) => {
6     return await apiClient.post(API_ENDPOINTS.AUTH.LOGIN, credentials);
7   },
8   register: async (userData) => {
9     return await apiClient.post(API_ENDPOINTS.AUTH.REGISTER, userData);
10  },
11   getMe: async () => {
12     return await apiClient.get(API_ENDPOINTS.AUTH.ME);
13   },
14 };
15
16 export default authService;
```

4.2.2. LoginForm.jsx & RegisterForm.jsx

- Chức năng: Xử lý nhập liệu từ người dùng.
- Công nghệ: Sử dụng react-hook-form để quản lý state của Form và antd để tạo giao diện UI.

```

17 const loginForm = ({ onLoginSuccess }) => { Show usages MichaelH271
18   const [loading, setLoading] = useState(initialState: false);
19   const [error, setError] = useState(initialState: null);
20   const { login } = useAuth();
21
22   const { control, ControlFieldValues, any, FieldValues } , handleSubmit : useFormHandleSubmitFieldValues, FieldValues } , formState: { errors : FieldErrorsFieldValues } } = useFo
23
24   const onSubmit = async (data) => {...};
25
26   return (
27     <form onSubmit={handleSubmit(onSubmit)}>
28       {error && <Alert message={error} type="error" showIcon style={{ marginBottom: '16px' }} />
29       <Space direction="vertical" style={{ width: '100%' }} size="Large">
30         <div>
31           <Text strong>Email</Text>
32           <Controller
33             name="email"
34             control={control}
35             render={({ field : ControllerRenderPropsFieldValues, TFieldValues extends any ? Pat... }) => (
36               <input {...field} placeholder="Enter your email" status={errors.email ? 'error' : ''} style={{ marginTop: '8px' }} />
37             )
38           />
39           {errors.email && <Text type="danger">{errors.email.message}</Text>}
40         </div>
41         <div>
42           <Text strong>Password</Text>
43           <Controller
44             name="password"
  
```

The screenshot shows a VS Code editor with the following details:

- File Explorer (Left):** Displays the project structure for 's3NgyuenVanAn18D04'. Key folders include 'src', 'assets', 'core', and 'components'. The 'components' folder is expanded, showing 'LoginForm.jsx' and 'RegisterPage.jsx'.
- Editor Tabs:** Multiple files are open, including 'Header.jsx', 'RoomFormModal.jsx', 'StaffLayout.jsx', 'useAuth.jsx', 'AuthContext.jsx', 'PrivateRoute.jsx', 'authService.jsx', 'LoginPage.jsx', and 'LoginForm.jsx' (the active file).
- Code in LoginForm.jsx:**

```

const LoginForm = ({ onLoginSuccess }) => {
  Show usages MichaelH271

  <form onSubmit={handleSubmit(onSubmit)}>
    <Space direction="vertical" style={{ width: '100%' }} size="large">
      <div>
        </div>
      </div>
      <div>
        <Text strong>Password</Text>
        <Controller
          name="password"
          control={control}
          render={({ field: ControllerRenderProps<FieldValues, TFieldValues extends any ? Path...> }) => (
            <Input.Password {...field} placeholder="Enter your password" status={errors.password ? 'error' : ''} style={{ marginTop: '8px' }} />
          )}
        />
        {errors.password && <Text type="danger">{errors.password.message}</Text>}
      </div>
      <Button type="primary" htmlType="submit" block loading={loading} size="large">
        Login
      </Button>
    </Space>
  </form>
);

export default LoginForm;
  
```
- Status Bar (Bottom):** Shows the current file path 's3NgyuenVanAn18D04 > src > features > auth > components > LoginForm.jsx' and the active editor 'LoginForm()'. It also displays the current time '10:25' and other system information.

```
1 > import { Typography, Grid } from '@mui/material';
2
3 const { Title, ForwardRefExoticComponent, Text } = Typography;
4
5 const { useBreakpoint } = Grid;
6
7 > const schema = yup.object().shape({
8   // 8 elements...
9 });
10
11 const RegisterForm = ({ onRegisterSuccess }) => {
12   Show usages MichaelH271
13   const [loading, setLoading] = useState(initialState: false);
14   const [error, setError] = useState(initialState: null);
15   const { login } = useAuth();
16   const screens = Partial<Record<...>> = useBreakpoint();
17
18   const { control, ControlFieldValues, any, FieldValues, handleSubmit, useFormHandleSubmit, FieldValues, FieldValues, formState: { errors: FieldErrors, FieldValues } } = useForm
19
20   const onSubmit = async (data) => {
21     Show usages MichaelH271
22     setLoading(value: true);
23     setError(value: null);
24     try {
25       const registerData = { email: data.email };
26
27       const response = await authService.register(registerData);
28       const token = typeof response === 'string' ? response : response.token;
29       if (!token) throw new Error('Registration failed, no token received');
30
31       const decoded = jwtPayload = decodeToken(token);
32       const id = decoded.customerID || decoded.customerId || decoded.id;
33       const userInfo = { id: id };
34
35       login(token, userInfo);
36     } catch (err) {
37       setError(value: err.response?.data?.message || 'Registration failed. Email might already exist.');
```

```
32 const RegisterForm = ({ onRegisterSuccess }) => {
33   Show usages MichaelH271
34   const onSubmit = async (data) => {
35     Show usages MichaelH271
36     const id = decoded.customerID || decoded.customerId || decoded.id;
37     const userInfo = { id: id };
38
39     login(token, userInfo);
40     if (onRegisterSuccess) onRegisterSuccess(userInfo);
41   } catch (err) {
42     setError(value: err.response?.data?.message || 'Registration failed. Email might already exist.');
```

1 Mar 21:41

Project

- src
 - assets
 - core
 - apiClient.js
 - axiosInstance
 - auth
 - AuthContext.js
 - authService.js
 - jwtUtils.js
 - useAuth.js
 - constants
 - utils
 - features
 - auth
 - components
 - LoginForm.js
 - RegisterForm.js
 - bookings
 - customers
 - profile
 - rooms
 - hooks
 - pages
 - auth
 - LoginPage.js
 - RegisterPage.js
 - customer
 - public
 - staff
 - routes
 - AppRouter.js
 - shared
 - components
 - common

```
32 const RegisterForm = ({ onRegisterSuccess }) => { Show usages Michaelht271
92 <form onSubmit={handleSubmit(onSubmit)}> Michaelht271, Today · finish as3
94 <Space direction="vertical" style={{ width: '100%' }} size="middle">
173 <div style={{ display: 'grid', gridTemplateColumns: screens.md ? '1fr 1fr' : '1fr', gap: '16px' }}>
186 <div>
187 <Text strong>Gender</Text>
188 <Controller
189   name="gender"
190   control={control}
191   render={({ field: ControllerRenderProps<FieldValues, TFieldValues extends any ? Pat...> }) => {
192     <Radio.Group {...field} style={{ display: 'block', marginTop: '8px' }}>
193       <Radio value="Male">Male</Radio>
194       <Radio value="Female">Female</Radio>
195       <Radio value="Other">Other</Radio>
196     </Radio.Group>
197   )}
198 </div>
199 {errors.gender && <Text type="danger">{errors.gender.message}</Text>}
200 </div>
201 <Button type="primary" htmlType="submit" block loading={loading} size="large" style={{ marginTop: '16px' }}>
202   Register
203 </Button>
204 </Space>
205 </form>
206 </div>
207 </div>
208 </div>
209 </div>
210 </div>
211 export default RegisterForm; Show usages Michaelht271
```

12 4 up-to-date Blame: Michaelht271 3/7/26, 8:29 PM 92.45 LF UTF-8 4 spaces

4.2. Customer Management

4.2.1. CustomerType

```
/** Michaelht271, Moments ago · finish as3
 * Customer Data Structure & Default Values based on Java Entity
 */
export const INITIAL_CUSTOMER_STATE = {
  telephone: '',
  emailAddress: '',
  customerBirthday: '',
  customerStatus: 1, // 1 for Active, 0 for Inactive
  password: '',
  roles: 'CUSTOMER'
};

export const CUSTOMER_STATUS_OPTIONS = [{label: string, value: number...} = [
  { label: 'Active', value: 1, color: 'green' },
  { label: 'Inactive', value: 0, color: 'red' },
];

export const MAP_CUSTOMER_TO_FORM = (customer) => ({
  customerID: customer.customerID,
  customerFullName: customer.customerFullName,
  telephone: customer.telephone,
  emailAddress: customer.emailAddress,
  customerBirthday: customer.customerBirthday,
  customerStatus: customer.customerStatus,
  roles: customer.roles
});
```

4.2.2. CustomerService

```
import ...  
  
const customerService: {} = {  
  getAll: async () => {  
    return await apiClient.get(API_ENDPOINTS.CUSTOMERS.BASE);  
  },  
  
  getById: async (id) => {  
    return await apiClient.get(API_ENDPOINTS.CUSTOMERS.DETAIL(id));  
  },  
  
  create: async (customerData) => {  
    return await apiClient.post(API_ENDPOINTS.CUSTOMERS.BASE, customerData);  
  },  
  
  update: async (id, customerData) => {  
    return await apiClient.put(API_ENDPOINTS.CUSTOMERS.DETAIL(id), customerData);  
  },  
  
  delete: async (id) => {  
    return await apiClient.delete(API_ENDPOINTS.CUSTOMERS.DETAIL(id));  
  },  
};  
  
export default customerService; Show usages Michaelht271
```

4.2.3. CustomerTable

```
> import ...  
  
const { Text : ForwardRefExoticComponent<...> } = Typography;  
  
const CustomerTable = ({ customers, loading, onEdit, onDelete, onView }) => {  
  const columns : [{title: string, dataIndex: st...} ] = [ 6 elements... ];  
  
  return (  
    <Table  
      columns={columns}  
      dataSource={customers || []}  
      rowKey="customerID"  
      loading={loading}  
      pagination={{ pageSize: 10 }}  
      style={{ borderRadius: '12px', overflow: 'hidden', boxShadow: '0 2px 8px rgba(0,0,0,0.05)' }}  
    />  
  );  
};  
  
export default CustomerTable;
```


4.2.4. CustomerFormModal

```
const CustomerFormModal = ({ open, onCancel, onFinish, editingCustomer, loading }) => {  
  <Modal  
    <Form  
      <Row gutter={16}>  
        <Col span={12}>  
          <Form.Item name="customerFullName" label="Full Name" rules={[{ required: true, message:  
        </Col>  
        <Col span={12}>  
          <Form.Item name="emailAddress" label="Email" rules={[{ required: true, type: 'email',  
        </Col>  
      </Row>  
  
      <Row gutter={16}>  
        <Col span={12}>  
          <Form.Item name="telephone" label="Telephone" rules={[{ required: true, message: 'Plea  
        </Col>  
        <Col span={12}>  
          <Form.Item name="birthday" label="Birthday"...>  
        </Col>  
      </Row>  
  
      <Form.Item name="customerStatus" label="Status" rules={[{ required: true }]}...>  
  
      <Form.Item style={{ textAlign: 'right', marginBottom: 0, marginTop: '24px' }}...>  
    </Form>  
  </Modal>  
);  
};  
  
export default CustomerFormModal; Show usages Michaelht271
```

4.2.5. CustomerPages

```
> import ...

const { Title : ForwardRefExoticComponent<...> } = Typography;

const CustomersPage = () => { Show usages  ⚡ Michaelht271
  const [customers : any[] , setCustomers] = useState( initialState: []);
  const [loading : boolean , setLoading] = useState( initialState: false);
  const [isModalOpen : boolean , setIsModalOpen] = useState( initialState: false);
  const [isDrawerOpen : boolean , setIsDrawerOpen] = useState( initialState: false);
  const [editingCustomer, setEditingCustomer] = useState( initialState: null);
  const [viewingCustomer, setViewingCustomer] = useState( initialState: null);

  const fetchCustomers = async () => { Show usages  ⚡ Michaelht271
    setLoading( value: true);
    try {...} catch (error) {
      message.error( content: 'Failed to fetch customers');
      setCustomers( value: []);
    } finally {
      setLoading( value: false);
    }
  };

  useEffect( effect: () => {...}, deps: []);

  const handleAdd = () => {...};

  const handleEdit = (record) => {...};

  const handleView = (record) => { Show usages  ⚡ Michaelht271
    setViewingCustomer(record);
```

```
const CustomersPage = () => { Show usages Michaelht271 1 3 ^ v

  return (
    <div>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', margin: '10px 0' }}>
        <Title level={2}>Customer Management</Title>
        <Button
          type="primary"
          icon={<PlusOutlined />}
          onClick={handleAdd}
          size="large"
          style={{ borderRadius: '8px' }}
        >
          Add Customer
        </Button>
      </div>

      <CustomerTable
        customers={customers}
        loading={loading}
        onEdit={handleEdit}
        onDelete={handleDelete}
        onView={handleView}
      />

      <CustomerFormModal
        open={isModalOpen}
        onCancel={() => setIsModalOpen( value: false)}
        onFinish={onFinish}
        editingCustomer={editingCustomer}
        loading={loading}
      />
    </div>
  )
}
```

```
const CustomersPage = () => { Show usages ⓘ Michaelht271
  <div>
    <Drawer
      <div>
        <div style={{ textAlign: 'center', marginBottom: '24px' }}>
          alignItems: 'center',
          justifyContent: 'center',
          margin: '0 auto',
          marginBottom: '12px'
        </div>
        {viewingCustomer.customerFullName.charAt(0).toUpperCase()}
      </div>
      <Title level={4}>{viewingCustomer.customerFullName}</Title>
      <Tag color={viewingCustomer.customerStatus === 1 ? 'green' : 'red'}>
        {viewingCustomer.customerStatus === 1 ? 'ACTIVE' : 'INACTIVE'}
      </Tag>
    </div>

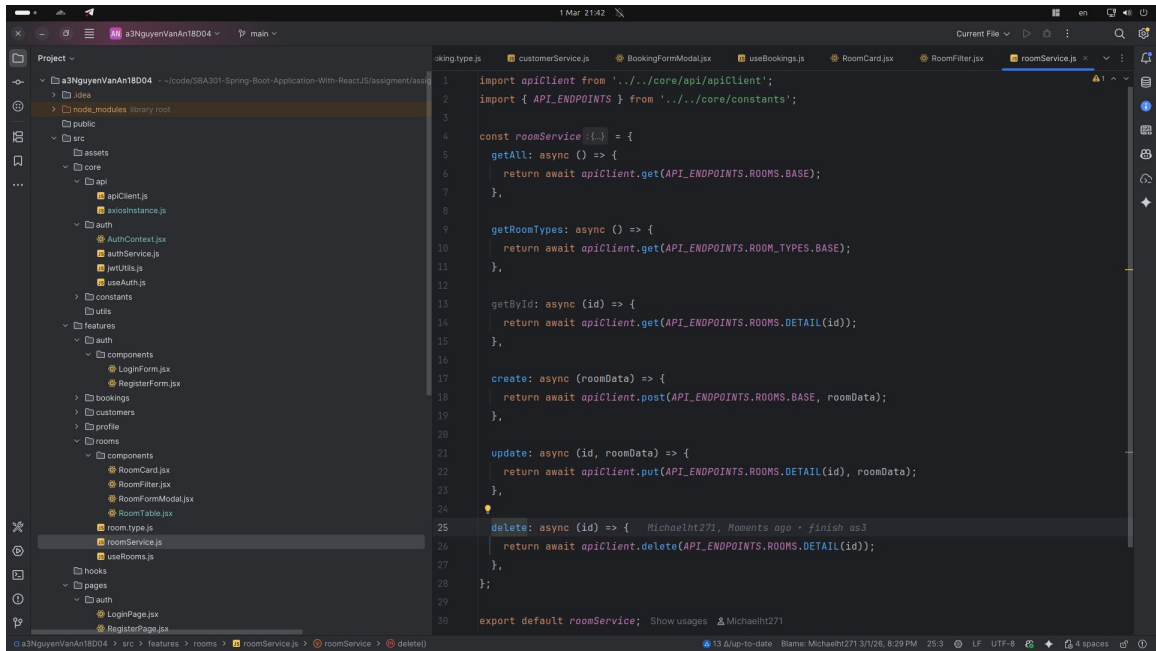
    <Divider />

    <Descriptions column={1} bordered labelStyle={{ background: '#fafafa', fontWeight: 'b'
      <Descriptions.Item label="Customer ID">#{viewingCustomer.customerId}</Descriptions.I
      <Descriptions.Item label="Full Name">{viewingCustomer.customerFullName}</Description
      <Descriptions.Item label="Email Address">{viewingCustomer.emailAddress}</Description
      <Descriptions.Item label="Phone Number">{viewingCustomer.telephone}</Descriptions.I
      <Descriptions.Item label="Date of Birth">{viewingCustomer.birthDay ? dayjs(viewingC
    </Descriptions>
  </div>
)}
</Drawer>
```

4.3. Feature: Quản lý Phòng (Room Management)

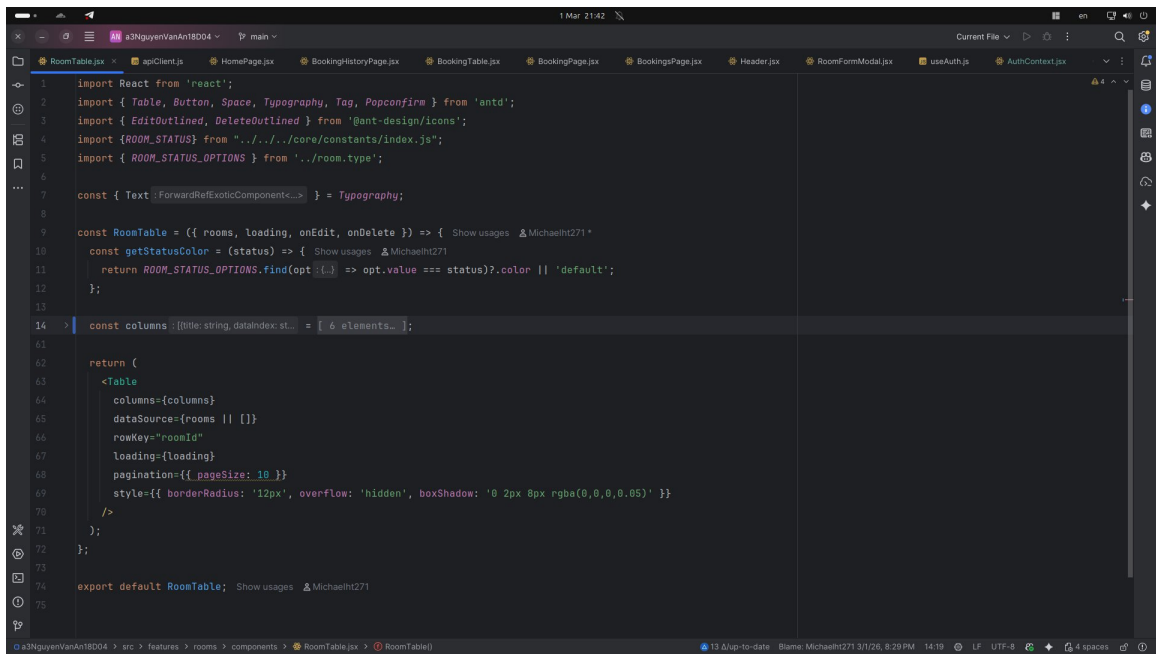
4.3.1. roomService.js

- Chức năng: Chứa các hàm getAllRooms, createRoom, updateRoom, deleteRoom.
- API: Gọi trực tiếp các Endpoints từ RoomInformationController của Backend.



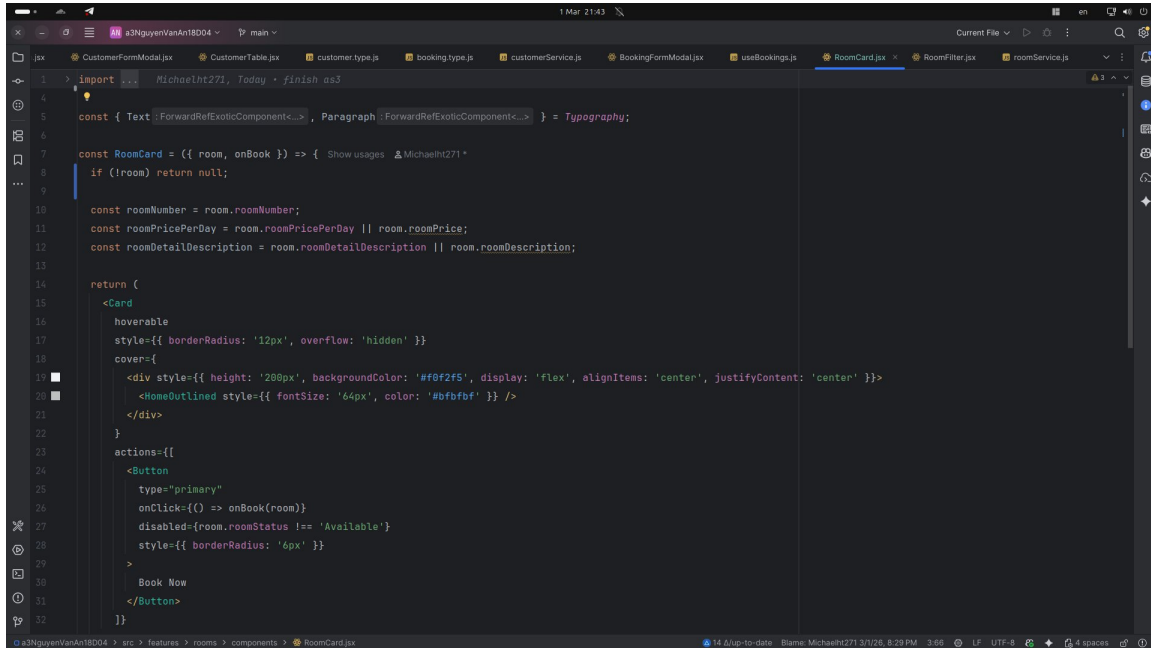
4.3.2. RoomTable.jsx (Staff View)

- Chức năng: Hiện thị danh sách phòng dưới dạng bảng cho Nhân viên.
- Tính năng: Tích hợp nút Sửa (Edit) và Xóa (Delete).



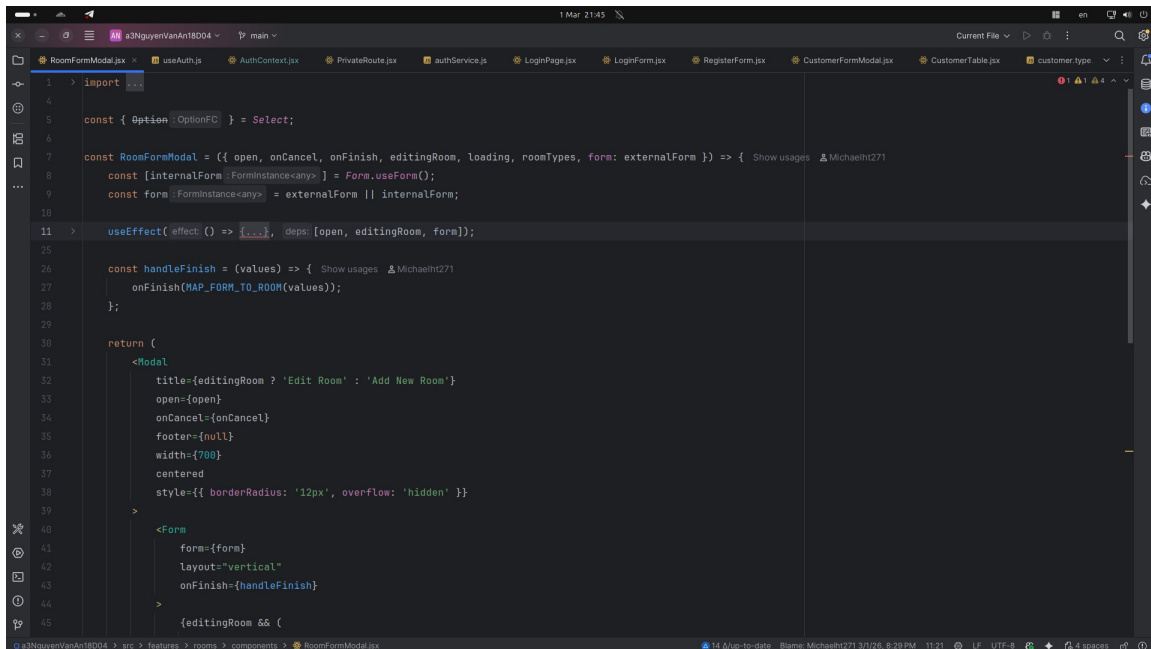
4.3.2. RoomCard.jsx & RoomFilter.jsx (Customer View)

- Chức năng: Hiển thị phòng dưới dạng thẻ trực quan và cung cấp bộ lọc tìm kiếm theo loại phòng hoặc giá tiền.

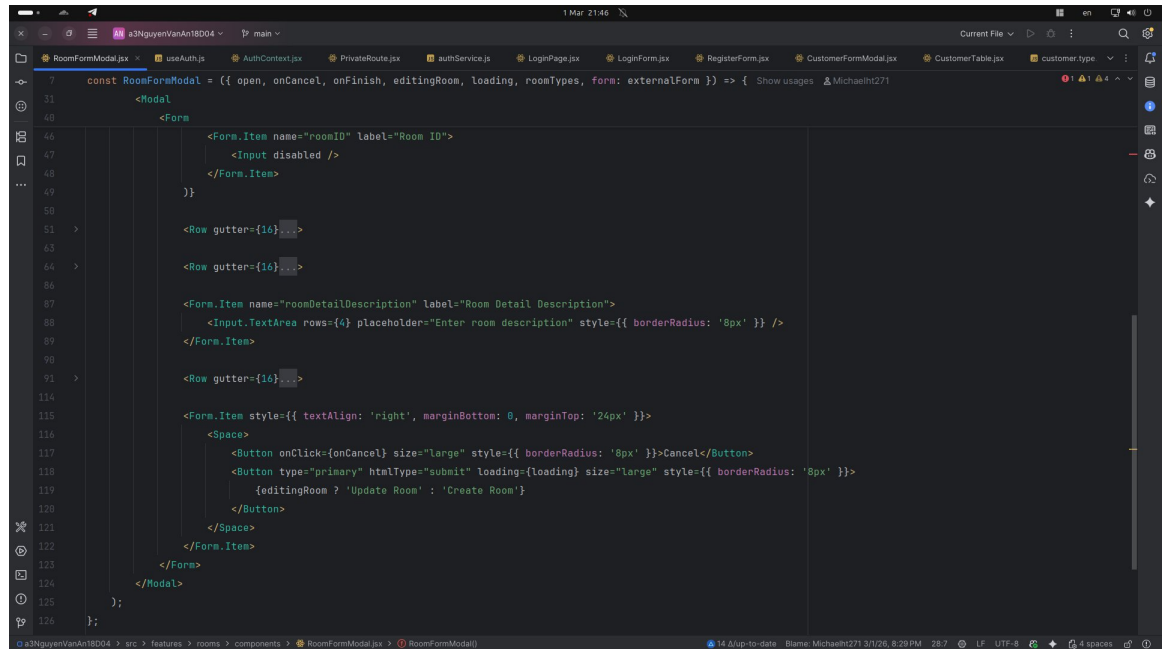


```
1 import { Typography } from '@mui/material';
2
3 const { Text } = ForwardRefExoticComponent<...>, Paragraph } = Typography;
4
5 const RoomCard = ({ room, onBook }) => { Show usages Michaelht271
6   if (!room) return null;
7
8   const roomNumber = room.roomNumber;
9   const roomPricePerDay = room.roomPricePerDay || room.roomPrice;
10  const roomDetailDescription = room.roomDetailDescription || room.roomDescription;
11
12  return (
13    <Card
14      hoverable
15      style={{ borderRadius: '12px', overflow: 'hidden' }}
16      cover={
17        <div style={{ height: '280px', backgroundColor: '#f0f2f5', display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
18          <HomeOutlined style={{ fontSize: '64px', color: '#bfbbf5' }} />
19        </div>
20      )
21      actions={
22        <Button
23          type="primary"
24          onClick={() => onBook(room)}
25          disabled={room.roomStatus !== 'Available'}
26          style={{ borderRadius: '6px' }}
27        >
28          Book Now
29        </Button>
30      )
31    </Card>
32  );
33 }
```

4.3.3. RoomFormModal

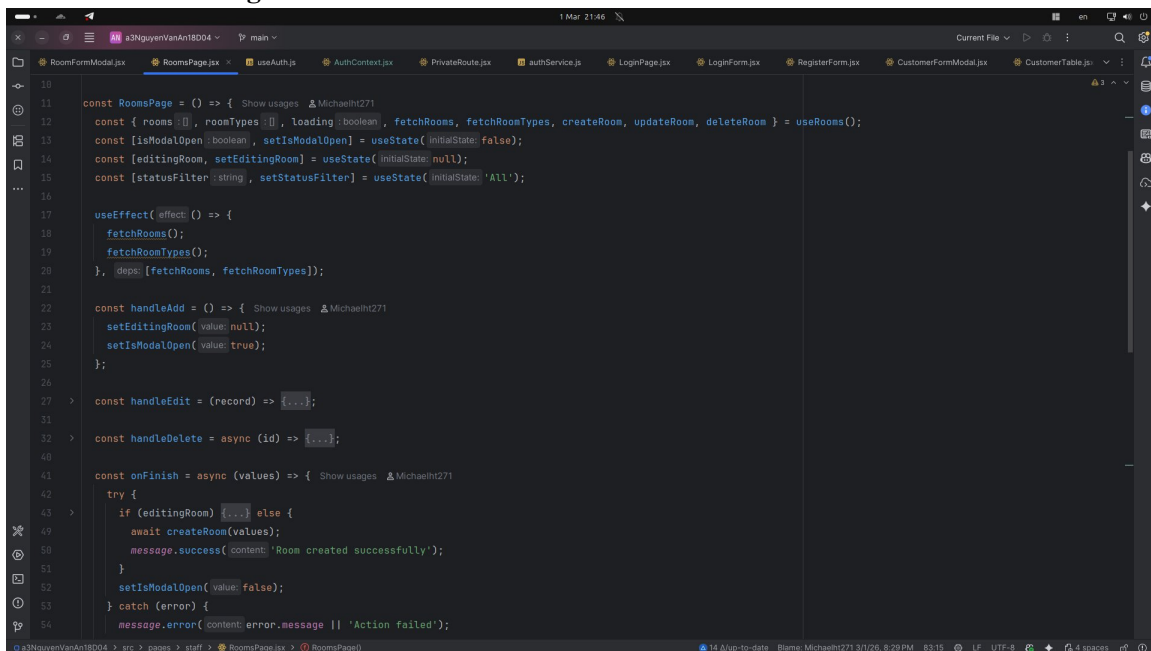


```
1 import { useState, useEffect } from 'react';
2
3 const { Option } = OptionFC } = Select;
4
5 const RoomFormModal = ({ open, onCancel, onFinish, editingRoom, loading, roomTypes, form: externalForm }) => { Show usages Michaelht271
6   const [internalForm, { FormInstance } = Form.useForm();
7   const form: FormInstance<any> = externalForm || internalForm;
8
9   useEffect(() => {
10     // ...
11   }, [open, editingRoom, form]);
12
13   const handleFinish = (values) => {
14     onFinish(MAP_FORM_TO_ROOM(values));
15   };
16
17   return (
18     <Modal
19       title={editingRoom ? 'Edit Room' : 'Add New Room'}
20       open={open}
21       onCancel={onCancel}
22       footer={null}
23       width={700}
24       centered
25       style={{ borderRadius: '12px', overflow: 'hidden' }}
26     >
27       <Form
28         form={form}
29         layout="vertical"
30         onFinish={handleFinish}
31       />
32     </Modal>
33   );
34 }
```



```
7 const RoomFormModal = ({ open, onCancel, onFinish, editingRoom, loading, roomTypes, form: externalForm }) => { Show usages MichaelH271
11   <Modal>
40     <Form>
46       <Form.Item name="roomID" label="Room ID">
47         <Input disabled />
48       </Form.Item>
49     </Form>
50   </Modal>
51   <Row gutter={16}>
52     <Form.Item name="roomDetailDescription" label="Room Detail Description">
53       <Input.TextArea rows={4} placeholder="Enter room description" style={{ borderRadius: '8px' }} />
54     </Form.Item>
55   </Row>
56   <Row gutter={16}>
57     <Form.Item style={{ textAlign: 'right', marginBottom: 0, marginTop: '24px' }}>
58       <Space>
59         <Button onClick={onCancel} size="large" style={{ borderRadius: '8px' }}>Cancel</Button>
60         <Button type="primary" htmlType="submit" loading={loading} size="large" style={{ borderRadius: '8px' }}>
61           {editingRoom ? 'Update Room' : 'Create Room'}
62         </Button>
63       </Space>
64     </Form.Item>
65   </Row>
66 </Modal>
67 </RoomFormModal>
68 </>
```

4.3.4. RoomPage



```
10 const RoomPage = () => { Show usages MichaelH271
11   const { rooms, roomTypes, loading, fetchRooms, fetchRoomTypes, createRoom, updateRoom, deleteRoom } = useRooms();
12   const [isModalOpen, setIsModalOpen] = useState(initialState: false);
13   const [editingRoom, setEditingRoom] = useState(initialState: null);
14   const [statusFilter, setStatusFilter] = useState(initialState: 'All');
15
16   useEffect(effect: () => {
17     fetchRooms();
18     fetchRoomTypes();
19   }, [deps: [fetchRooms, fetchRoomTypes]]);
20
21   const handleAdd = () => { Show usages MichaelH271
22     setEditingRoom(value: null);
23     setIsModalOpen(value: true);
24   };
25
26   const handleEdit = (record) => {...};
27
28   const handleDelete = async (id) => {...};
29
30   const onFinish = async (values) => { Show usages MichaelH271
31     try {
32       if (editingRoom) {...} else {
33         await createRoom(values);
34         message.success(content: 'Room created successfully');
35       }
36       setIsModalOpen(value: false);
37     } catch (error) {
38       message.error(content: error.message || 'Action failed');
39     }
40   }
41 }
```

4.4. Feature: Quản lý Đặt phòng (Booking Management)

4.4.1. bookingService.js

- Chức năng: Xử lý logic đặt phòng (createBooking) và lấy lịch sử (getBookingHistory).


```

import ...

const bookingService :{...} = {
  getAll: async () => {
    return await apiClient.get(API_ENDPOINTS.BOOKINGS.BASE);
  },

  getById: async (id) => {
    return await apiClient.get(API_ENDPOINTS.BOOKINGS.DETAIL(id));
  },

  getDetails: async (bookingId) => {
    return await apiClient.get(API_ENDPOINTS.BOOKINGS.DETAILS(bookingId));
  },

  getByCustomer: async (customerId) => {
    return await apiClient.get(API_ENDPOINTS.BOOKINGS.BY_CUSTOMER(customerId));
  },

  create: async (bookingData) => {
    return await apiClient.post(API_ENDPOINTS.BOOKINGS.BASE, bookingData);
  },

  update: async (id, bookingData) => {
    return await apiClient.put(API_ENDPOINTS.BOOKINGS.DETAIL(id), bookingData);
  },

  updateStatus: async (id, status) => {
    return await apiClient.put(API_ENDPOINTS.BOOKINGS.UPDATE_STATUS(id), { data: { status } });
  },
}

```

4.4.2. BookingFormModal.jsx

- Chức năng: Form cho phép khách hàng chọn ngày nhận phòng, ngày trả phòng và chọn nhiều phòng cùng lúc.
- Logic: Tự động tính toán tổng tiền dựa trên số ngày và đơn giá phòng.


```

> import ...

const { Option : OptionFC } = Select;
const { RangePicker : import("react").ForwardRefExot... } = DatePicker;

const BookingFormModal = ({ open, onCancel, onFinish, editingBooking, loading, rooms, customers }) {
  const [form : FormInstance<any> ] = Form.useForm();

  useEffect( effect: () => {
>    if (open) {...}
  }, deps: [open, editingBooking, form]);

  const handleValuesChange = (changedValues, allValues) => { Show usages  ⚡ Michaelht271
    const { roomId, dates } = allValues;
>    if (roomId && dates && dates[0] && dates[1]) {...}
  };

  const handleFinish = (values) => { Show usages  ⚡ Michaelht271
    onFinish(MAP_FORM_TO_BOOKING(values));
  };

  return (
    <Modal
      title={editingBooking ? 'Edit Booking' : 'Create New Booking'}
      open={open}
      onCancel={onCancel}
      footer={null}
      width={700}
      centered
      style={{ borderRadius: '12px', overflow: 'hidden' }}

```

```

const BookingFormModal = ({ open, onCancel, onFinish, editingBooking, loading, rooms, ...props }) => {
  <Modal
    style={{ borderRadius: '12px', overflow: 'hidden' }}
  >
    <Form
      form={form}
      layout="vertical"
      onFinish={handleFinish}
      onValuesChange={handleValuesChange}
    >
      <Row gutter={16}>
        <Col span={12}>
          <Form.Item name="customerID" label="Customer" rules={[{ required: true, message: 'Please enter customer ID' }]}>
            <Input type="text" value={customerID} onChange={handleChange} />
          </Form.Item>
        </Col>
        <Col span={12}>
          <Form.Item name="roomID" label="Room" rules={[{ required: true, message: 'Please select a room' }]}>
            <Select value={roomID} onChange={handleChange} options={rooms} />
          </Form.Item>
        </Col>
      </Row>

      <Row gutter={16}>
        <Col span={12}>
          <Form.Item name="dates" label="Start & End Dates" rules={[{ required: true, message: 'Please select dates' }]}>
            <DatePicker value={dates} onChange={handleChange} />
          </Form.Item>
        <Col span={12}>
          <Form.Item name="bookingStatus" label="Status" rules={[{ required: true, message: 'Please select status' }]}>
            <Select value={bookingStatus} onChange={handleChange} options={bookingStatusOptions} />
          </Form.Item>
        </Col>
      </Row>

      <Row gutter={16}>
        <Col span={24}>
          <Form.Item name="comment" label="Comment" rules={[{ required: false, message: 'Optional comment' }]}>
            <TextArea value={comment} onChange={handleChange} />
          </Form.Item>
        </Col>
      </Row>
    </Form>
  </Modal>
}

```

```

const BookingFormModal = ({ open, onCancel, onFinish, editingBooking, loading, rooms, ...props }) => {
  <Modal
    <Form
      <Row gutter={16}>
        </Col>
        <Col span={12}>
          <Form.Item name="bookingStatus" label="Status" rules={[{ required: true }]}>
            </Col>
          </Row>

          <Row gutter={16}>
            <Col span={24}>
              <Form.Item name="totalPrice" label="Total Price (VND)" rules={[{ required: true }]}>
                </Col>
              </Row>

              <Form.Item style={{ textAlign: 'right', marginBottom: 0, marginTop: '24px' }}>
                <Space>
              </Form.Item>
            </Form>
          </Modal>
        );
      };
    export default BookingFormModal;
  
```

4.4.3. BookingTable.jsx (Staff View)

- Chức năng: Cho phép nhân viên quản lý trạng thái các đơn đặt phòng (Confirm, Check-in, Check-out).

```

import React from 'react';
import { Table, Button, Space, Typography, Tag, Select } from 'antd';
import { InfoCircleOutlined, EditOutlined } from '@ant-design/icons';
import dayjs from 'dayjs';
import { BOOKING_STATUS_CONFIG } from '../booking.type';

const { Text : ForwardRefExoticComponent<...> } = Typography;
const { Option : OptionFC } = Select;

const BookingTable = ({ bookings, loading, onDetail, onEdit, onStatusChange, userRole }) => {
  const getStatusColor = (status) => {
    return BOOKING_STATUS_CONFIG[status?.toUpperCase()]?.color || 'default';
  };

  const columns = [...].filter(col => !col.hidden);

  return (
    <Table
      columns={columns}
      dataSource={bookings || []}
      rowKey="bookingReservationID"
      loading={loading}
      pagination={{ pageSize: 10 }}
      style={{ borderRadius: '12px', overflow: 'hidden', boxShadow: '0 2px 8px rgba(0,0,0,0.05)' }}
    />
  );
};

export default BookingTable;

```

4.4.4. BookingHistoryPage.jsx

- Chức năng: Hiển thị danh sách các lần đặt phòng trước đây của chính khách hàng đó.

```

const BookingHistoryPage = () => { Show usages Michaelht271
  <div style={{ padding: '24px 0' }}>
    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
      <Text type="secondary">Review your previous and upcoming stays at FUMiniHotel</Text>
    </div>
  </div>

  <Divider />

  <BookingTable
    bookings={bookings}
    loading={loading}
    onDetail={handleVi
    userRole={ROLES.CU
  />

  <BookingDetailModal
    open={isDetailOpen}
    onCancel={() => setIsDetailOpen(value: false)}
    booking={viewingBooking}
    userRole={ROLES.CUSTOMER}
    onStatusChange={handleCancel}
  />
</div>
);
};

export default BookingHistoryPage; Show usages Michaelht271

```

```
const bookings: []
```

```
src/.../customer/BookingHistoryPage.
jsx
```

4.5. Profile Page

```
const ProfilePage = () => { Show usages Michaelht271
  useEffect(effect: () => {
    fetchProfile(customerId).then(data => [...]),
  },
  }, deps: [user, fetchProfile, form]);

const onFinish = async (values) => { Show usages Michaelht271
  try {...} catch (error) {
    console.error('Update profile error:', error);
    message.error({content: error.message || 'Failed to update profile'});
  }
};

return (
  <div style={{ maxWidth: '1000px', margin: '0 auto', padding: '24px' }}>
    <Title level={2}><UserOutlined /> My Profile</Title>
    <Divider />
    <Row gutter={[24, 24]}>
      <Col>
        <RowProps
          gutter?: number | string | Partial<Record<"xxl" | "xxl" | "xl" | "lg" | "md" | "sm" | "xs", number>> |
          [number | string | Partial<Record<"xxl" | "xxl" | "xl" | "lg" | "md" | "sm" | "xs", number>>, number |
          string | Partial<Record<"xxl" | "xxl" | "xl" | "lg" | "md" | "sm" | "xs", number>>]
        </Col>
        <Col>
          <ProfileForm form={form} onFinish={onFinish} loading={loading} />
        </Col>
      </Row>
    </div>
  );
};
```

4.6. Other

4.6.1. Header

```
const Header = ({ layoutType :string = 'public' }) => {  
  <AntHeader style={{display: 'flex'...}}>  
    <div className="logo" style={{color: 'white'...}}>  
      <HomeOutlined style={{ marginRight: '8px' }} />  
      FUMiniHotel  
    </Link>  
  </div>  
  
  <Menu  
    theme="dark"  
    mode="horizontal"  
    selectedKeys={[location.pathname]}  
    items={getMenuItems()}  
    style={{ flex: 1, minWidth: 0, borderBottom: 'none' }}  
  />  
  
  <div style={{ display: 'flex', alignItems: 'center', marginLeft: '16px' }}>  
    {isAuthenticated ? (  
      <Space size="middle">  
        <div style={{ display: 'flex', flexDirection: 'column', lineHeight: '1.2', alignItems: 'center' }}>  
          <Button  
            type="primary"  
            danger  
            icon={<LogoutOutlined />}  
            onClick={logout}  
            style={{ borderRadius: '6px' }}  
          ...>  
        </Space>  
      ) : (  
        <Space...>
```

4.6.2. Footer

```
import ...

const { Footer: AntFooter : ForwardRefExoticComponent<...> } = Layout;
const { Text : ForwardRefExoticComponent<...> , Title : ForwardRefExoticComponent<...> , Link : ForwardRefExoticComponent<...> } = Text;

const Footer = () => { Show usages Michaelht271
  return (
    <AntFooter style={{
      backgroundColor: '#f0f2f5',
      padding: '48px 50px', Michaelht271, Today * finish as3
      color: '#595959',
      boxShadow: '0 -2px 8px rgba(0,0,0,0.05)'
    }}>
      <Row gutter={[32, 32]}>
        <Col xs={24} md={8}>
          <Title level={4} style={{ marginBottom: '24px' }}>
            <HomeOutlined style={{ marginRight: '8px' }} />
            FUMiniHotel
          </Title>
          <Paragraph>
            The best hotel management system for FPT University students.
            Book your rooms quickly and securely with our mini hotel system.
          </Paragraph>
          <Space size="large" style={{ fontSize: '20px', marginTop: '16px' }}>
            <Link href="#"><FacebookOutlined /></Link>
            <Link href="#"><TwitterOutlined /></Link>
            <Link href="#"><InstagramOutlined /></Link>
          </Space>
        </Col>
      </Row>
    </AntFooter>
  )
}
```



```
const Footer = () => { Show usages ⓘ Michaelht271 *
  <AntFooter style={{
    <Row gutter={[32, 32]}>
      <Col xs={24} md={8}>
        <Space direction="vertical" style={{ width: '100%' }}>
          </Space>
        </Col>
      </Row>

      <Divider style={{ margin: '32px 0' }} />

      <div style={{ textAlign: 'center' }}>
        <Text type="secondary">
          FUMiniHotel ©{new Date().getFullYear()} Created by a3NguyenVanAn18D04
        </Text>
      </div>
    </AntFooter>
  )};
};

const Paragraph = ({ children }) => ( Show usages ⓘ Michaelht271
  <div style={{ marginBottom: '16px', lineHeight: '1.6' }}>
    <Text type="secondary">{children}</Text>
  </div>
);

export default Footer; Show usages ⓘ Michaelht271
```